

Depots: Theft-Proof, Self-Custodial Bitcoin For Billions Of Users

John Law

September 13, 2026

Version 1.0

Abstract

In order to allow many billions of users to safely own, send and receive bitcoin, a scaling protocol must:

1. have a tiny on-chain footprint (e.g., 1 or 2 vbytes/user/year),
2. allow users to participate even if they don't own enough bitcoin to pay the fee for putting a transaction on-chain, and
3. avoid creating more off-chain transactions than can be put on-chain before theft occurs.

No trust-free scaling protocol meets these requirements, so it's widely believed that most users will have to rely on custodial, trust-based solutions such as exchanges.

This paper presents a protocol for *depots* that meets all three of the above requirements. A single depot allows many thousands or millions of users to own bitcoin in off-chain Lightning channels.

Depots rely on *griever penalization* for their security. In a depot a large number of users partner with an operator, with neither the users nor the operator being able to steal funds. The users can force the operator to lose some funds (and vice versa), but only by losing a proportional expected amount of their own funds. As a result, someone who only pairs with self-interested parties will never be grieved.

Depots require the CTV (CheckTemplateVerify) **[BIP-119]** and CSFS (CheckSigFromStack) **[BIP-348][Roose25]** enhancements to the Bitcoin protocol. Depots are much more efficient if the PC (PairCommit) **[BIP-PC][Black24]**, multiply (OP_MUL) **[Russell24]** and mod (OP_MOD) **[Russell24]** operators are supported, but they don't require those operators.

1 Depot Overview

In order to create a *depot*, an operator puts a Funding transaction with a single Taproot output on-chain. This output can be used to create probabilistic off-chain Lightning channels for many thousands or millions of users. The output's internal key is the root of a merkle tree containing the parameters of the depot and the root of a merkle-sum tree **[Osunto21]** that specifies the users of the depot and the allocation of a maximum number of channels to each user.

To acquire bitcoin in a depot, a user makes a Lightning payment to the operator to purchase a number of channels not exceeding the user's maximum allocation. Each channel is owned by the user and the operator and has a $1/P$ chance of being a *hit*, where P is a large prime (e.g., between a hundred thousand and a billion). The channel can only be put on-chain if it's a hit, but if it is put on-chain it pays the user P times as much as the user paid for the channel.

Each depot has a predefined expiry and the depot's channels can't be put on-chain until after the expiry. Users should drain their funds prior to the expiry, either by making Lightning payments or by transferring their funds to another depot. After the expiry, the operator claims all funds in the depot unless a user who didn't drain has a hit. If exactly one channel is a hit, the user owning that channel can put it on-chain, thus splitting the depot's funds between the user and the operator. On the other hand, if more than one channel is a hit, all funds in the depot are burned.

Depots combine 3 key ideas:

1. a time-limited output that's funded and put on-chain by an operator and supports a large number of users (as in Timeout Trees **[Law21][Law23b]**, Ark **[Keceli23]** or SuperScalar **[ZmnSC24]**),
2. randomization that replaces a small deterministic output with a much larger probabilistic output having the same expected value **[Rivest97][Cal12]**, and
3. griever penalization to encourage cooperative behavior **[Riard22][Law24][Law25]**.

These ideas work together:

- the use of an operator eliminates the need to coordinate large numbers of users, as each user only interacts with the operator,
- the use of a time limit allows users to obtain Lightning channels that never have to be put on-chain (as drained channels can be reclaimed by the operator without putting them on-chain),
- randomization allows users who have insufficient funds for putting a transaction on-chain to (probabilistically) obtain high-value transactions that can be put on-chain,
- randomization results in a small constant-size on-chain footprint per depot even in the worst case (thus supporting an unbounded number of users per depot and eliminating the risk of a "thundering herd" **[Towns23]** of users attempting to go on-chain if many depots reach their expiry without users draining their funds),

- griever penalization incentivizes users to drain their funds (and the operator to cooperate in draining those funds) before the expiry,
- griever penalization penalizes an operator who attempts to steal funds by putting the wrong transactions on-chain, and
- griever penalization allows Lightning sends and receives to be resolved entirely off-chain.

Depots are a remarkably scalable way of supporting users who own small amounts of bitcoin. For example, it will be shown below that depots require at most a small fraction of the block space to create Lightning channels for 10 billion users, where:

- each user owns a Lightning channel with 10 different operators, and
- the users send and receive hundreds of thousands of payments per second to and from those Lightning channels¹.

Throughout the remainder, let U be a user and O be the operator.

2 Security

This section analyses the security of the Bitcoin, Lightning and depot protocols.

2.1 Bitcoin's Security

Even Bitcoin's security relies on the self-interest of miners [Nak09]:

If a greedy attacker is able to assemble more CPU power than all the honest nodes, he would have to choose between using it to defraud people by stealing back his payments, or using it to generate new coins. He ought to find it more profitable to play by the rules, such rules that favour him with more new coins than everyone else combined, than to undermine the system and the validity of his own wealth.

In practice, miners have exhibited this self-interest for many years. As a result, self-custodied bitcoin is rightly viewed as being very secure.

2.2 Lightning's Security

A Lightning user always has transactions which give them their full channel payment, minus fees, provided those transactions can be put on-chain within a predefined time window. The security of Lightning thus depends on the relative value of the funds being secured and the fees required for putting transactions on-chain that claim those funds.

1 In addition to the users' Lightning channels, dedicated users will need to create routing channels to support the depot users' Lightning payments. While depots could theoretically be used to create those routing channels, timeout trees are better suited to creating them. As will be shown in Section 5.5, these routing channels should require at most a small fraction of the block space.

2.2.1 Funds Greatly Exceed Fees

If the value of a Lightning channel or payment greatly exceeds the cost of claiming that channel or payment on-chain, Lightning maintains most of Bitcoin's security properties. The main additional risk comes from the requirement that the Lightning user must put their transactions on-chain within a predefined window. The length of this window is given by a security parameters (e.g., *to_self_delay* or *cltv_expiry_delta*) that trades off security against latency in accessing one's funds. As long as the security parameters are set conservatively, Lightning balances and payments remain secure.

2.2.2 Funds Equal Fees

If the value of a Lightning channel or payment equals the cost of claiming that channel or payment on-chain, Lightning's security properties depend on the behavior of the channel partners. In particular, if Bob is forced by Alice to go on-chain to claim his Lightning balance or payment, and if Bob has to pay the on-chain fees, Bob will have to pay the entire value of his balance or payment to claim it, thus losing his funds. Note that Alice loses none of her funds by performing this attack. Despite the lack of any explicit penalty, such griefing attacks are rare in practice **[Harding24]** because Alice doesn't benefit from the attack and her reputation is damaged. As a result, Lightning payments that are comparable to the on-chain cost of securing the payment are made routinely and are secure.

2.2.3 Funds Smaller Than Fees

If the value of a Lightning payment is smaller than the cost of claiming the payment on-chain, choosing to claim the payment results in a net loss to the party claiming it. Therefore, a self-interested party may be expected to forgo claiming the payment. Also, note that in this case the attacker has a potential self-interest in performing the attack, because they may gain the value of the payment (due to its not being claimed on-chain). Such an attack will be called a *bullying attack*.

In reality, parties do choose to go on-chain to claim Lightning payments, even when the cost of claiming the payment exceeds the value of the payment. This is because a party who succumbs to a bullying attack proves that they can be victimized by similar attacks in the future. Furthermore, standard Lightning wallet software claims the payment on-chain (for the reason given above) **[Moreh24]**, so in practice users of such software don't make a conscious decision to accept a one-time loss in order to preserve their reputation. However, even if a conscious decision were required, there are strong human emotions that protect people against bullying. No one wants to be seen as a sucker who can be taken advantage of. This has been demonstrated with the Ultimatum Game **[Ultimatum]** in which, even when presented with a one-time opportunity to obtain free money, many users will forgo that money if they feel that accepting it would allow another party to take advantage of them. Another way of explaining why users resist bullying is that refusing to be bullied is in one's long-term interest (because one's reputation is preserved), despite the short-term costs.

Because Lightning users refuse to be bullied and their partners know they will refuse to be bullied, Lightning payments are routinely made that are smaller than the on-chain cost of claiming them.

2.2.4 Conclusions For Lightning

In conclusion, there's extensive practical evidence that Lightning payments are secure, even when that security depends on the parties acting in their own (long-term) self-interest. Specifically, there doesn't appear to be evidence of Lightning users performing griefing attacks, despite there being are no financial penalties for such attacks.

2.3 Depot's Security

A depot's security relies on users putting transactions on-chain within predefined time windows and on the parties acting in their own (long-term) self-interest. As a result, the observed security of the Lightning protocol (even when fees exceed payment amounts) is strong evidence for the security of depots.

2.3.1 Quantified Griefer Penalization

In fact, unlike the Lightning protocol, depots penalize any party that performs a griefing attack. Specifically, each depot defines two griefing security parameters, GrO and GrU . At any give time, let

- $LossO$ = the operator's expected loss and
- $LossU$ = the sum of the expected losses of all users.

It will be required that at all times:

- $\frac{LossO}{LossU} \geq GrO$ and
- $\frac{LossU}{LossO} \geq GrU$

where $0 < GrO$, $0 < GrU$ and $GrO * GrU < 1$. For example, a value of $GrU = 0.30$ means that users who perform a griefing attack must expect to lose at least 30% as much as the operator loses.

These security parameters are set large enough to discourage both the operator and the users from griefing each other. Note that O devotes significant resources by funding the depot and O can create a series of depots, thus building a reputation for following the depot protocol. In contrast, users may be new to Bitcoin and they do not fund depots. As a result, it's likely that GrO will be smaller than GrU in practice². Also, note that even very small values of GrO and GrU should be effective, as any positive value will deter a self-interested party from performing a griefing attack³.

² Values of GrO up to 0.10 and GrU up to 0.50 yield relatively efficient depots.

³ Note that a protocol that allows O to steal funds from users can be viewed as having a GrO security parameter of -1, as O can cause a loss of X to users while gaining X (or equivalently, "losing" $-X$). Similarly, Lightning payments that are smaller than the on-chain fees required to secure them can be viewed as having a griefing security parameter of 0.

2.3.2 The Off-Chain Lightning Protocol And The Depot Protocol

Depots use two separate protocols for securing users' Lightning payments and balances, both of which rely on griever penalization.

First, the off-chain Lightning protocol uses temporary burn outputs within Lightning channels in order to allow users to make sends and receives. The security of the off-chain Lightning protocol is quantified and examined in Appendix F.

Second, the depot protocol allows users to acquire and update Lightning channels while staying off-chain in the normal case. The security of the depot protocol is quantified and studied in Appendices B through E.

3 Depot Protocol For Creating And Revoking Channels

This section presents the protocol for creating and revoking channels in a depot. Before presenting the actual protocol, a simplified version is given. This simplified protocol lacks important features and has security vulnerabilities, but is included in order to give intuition into the actual protocol's operation.

3.1 Simplified Depot Protocol

3.1.1 Purchasing And Revoking Channels

In order to deposit funds in a depot, U purchases one or more off-chain Lightning channels from O. To purchase channel i , U makes a *guess* G_i that's a number in the range $0..P-1$ and gets O's signature for that guess. When getting O's signature, U doesn't reveal G_i to O. Instead, U selects a *salt value* $Salt_i$ that's hashed together with G_i (using the PC operator⁴) before getting O's signature, thus preventing O from learning G_i .

After the expiry, O reveals the depot's *target* t , which is a number in the range $0..P-1$. (If O fails to reveal a target within a fixed window of time, a default target is used.) A channel is a hit if the user's guess for that channel matches the target value.

Once U has purchased channel i , U can use channel i to send and receive bitcoin. If U and O follow the depot protocol, U will drain the channel before the depot's expiry by making payments from U and by moving U's funds to a channel in another depot. As part of draining channel i , U will *revoke* channel i by revealing G_i and $Salt_i$ to O, thus allowing O to select a target that differs from the channel's guess and eliminating the possibility of the channel being a hit. A channel i is said to be *active* from the time U gets O's signature for guess G_i until U revokes the channel. Note that O can't sell an unbounded number of channels, because when channels are revoked they could reveal guesses for every possible target value, at which point it would be impossible for O to select a target that doesn't hit a revoked channel.

4 Implementation without using the PC operator is presented in Appendix H.2.

In order to purchase a channel that pays X sats to U , U makes a Lightning payment of X/P sats to O . Thus, a channel's cost exactly equals its expected value to U if it's the only channel that's active after the depot's expiry, while a channel's cost is an upper bound on its expected value to U if there are multiple active channels after the depot's expiry⁵. If an active Lightning channel pays X sats to U when put on-chain, U 's *balance in the Lightning channel* is X/P . U 's *balance in the depot* equals the total of U 's balances in all of U 's active channels. Finally, if an active Lightning channel pays Y sats to O when put on-chain, O 's *balance in the Lightning channel* is Y/P .

3.1.2 Transactions

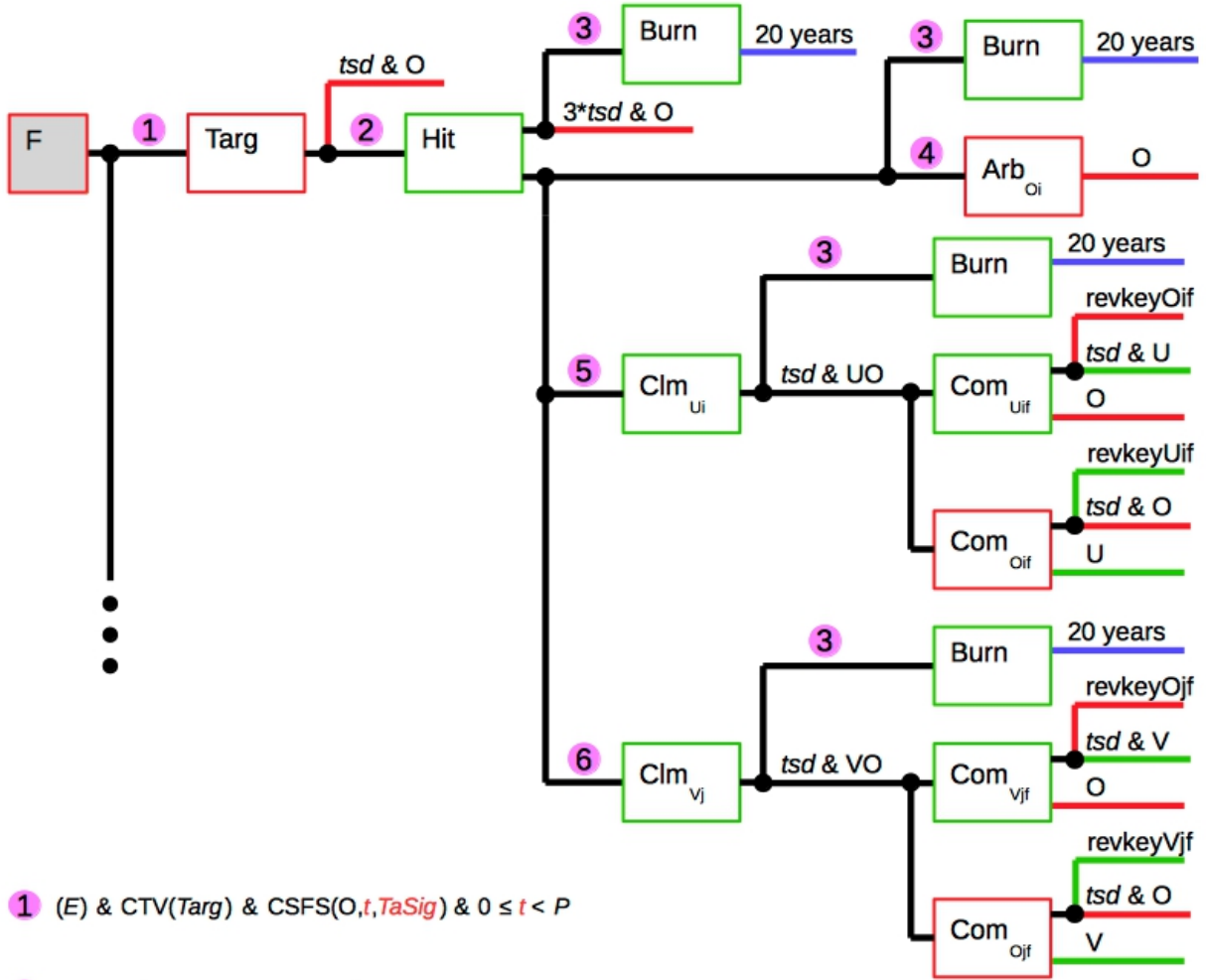
The on-chain transactions that implement the simplified protocol for creating and revoking channels are shown in Figures 1 and 2 below. Figure 1 shows the transactions put on-chain when O provides the target value t , and Figure 2 shows the transactions put on-chain when O fails to provide such a target value within the required time window.

In these figures and in figures throughout this paper, black values come from the tapleaf script and red values come from the witness stack. Each transaction output shows the conditions required for spending the output. The first condition, if required, is a relative delay of one or more *to_self_delay* (*tsd*) safety parameters or 20 years (for a burn), or a parenthesized absolute delay until the expiry (E) of the depot. In addition, outputs show the user(s) (if any) who must sign for the child transaction (e.g., user U or V , operator O , or both U and O , or the owner of a Lightning channel's revocation key).

The notation $CTV(X)$ denotes that the CTV operator forces the child transaction to be X , where X is either fixed in advance by the parent transaction (in which case the value X comes from the tapleaf script and is shown as black) or selected dynamically by the party creating the child transaction (in which case the value X comes from the witness stack and is shown as red). The notation $CSFS(O, \langle X_1, X_2, \dots, X_n \rangle, Sig)$ denotes that the CSFS operator requires O 's signature (Sig) for the value $PC(X_1, PC(X_2 \dots PC(X_{n-1}, X_n)))$ where PC is the PairCommit operator⁶.

5 The existence of multiple active channels creates a chance of more than one hit, thus causing the depot's funds to be burned.

6 Implementation without using the PC operator is discussed in Appendix H.2.



- 1 (E) & CTV(Targ) & CSFS(O,t,TaSig) & $0 \leq t < P$
- 2 CTV(Hit) & CSFS(O,t,TaSig) &
CSFS(O,<Clm_Ui,Clm'_Ui,i,Salt_i,G_j>,GuSig_i) & $G_j = t$
- 3 CTV(Burn) & CSFS(O,t,TaSig) & $GuSig_i \neq GuSig_j$ &
CSFS(O,<Clm_Ui,Clm'_Ui,i,Salt_i,G_j>,GuSig_i) & $G_j = t$ &
CSFS(O,<Clm_Vj,Clm'_Vj,j,Salt_j,G_j>,GuSig_j) & $G_j = t$
- 4 tsd & CTV(Arb_Oi) & CSFS(O,t,TaSig) &
CSFS(O,<Arb_Oi,Arb'_Oi,i,ArbSalt_i,G_j>,ArbGuSig_i) & $G_j = t$
- 5 tsd & CTV(Cl_m_Ui) & CSFS(O,t,TaSig) &
CSFS(O,<Clm_Ui,Clm'_Ui,i,Salt_i,G_j>,GuSig_i) & $G_j = t$
- 6 tsd & CTV(Cl_m_Vj) & CSFS(O,t,TaSig) &
CSFS(O,<Clm_Vj,Clm'_Vj,j,Salt_j,G_j>,GuSig_j) & $G_j = t$

Figure 1. Normal resolution of a simplified depot using an operator-selected target.

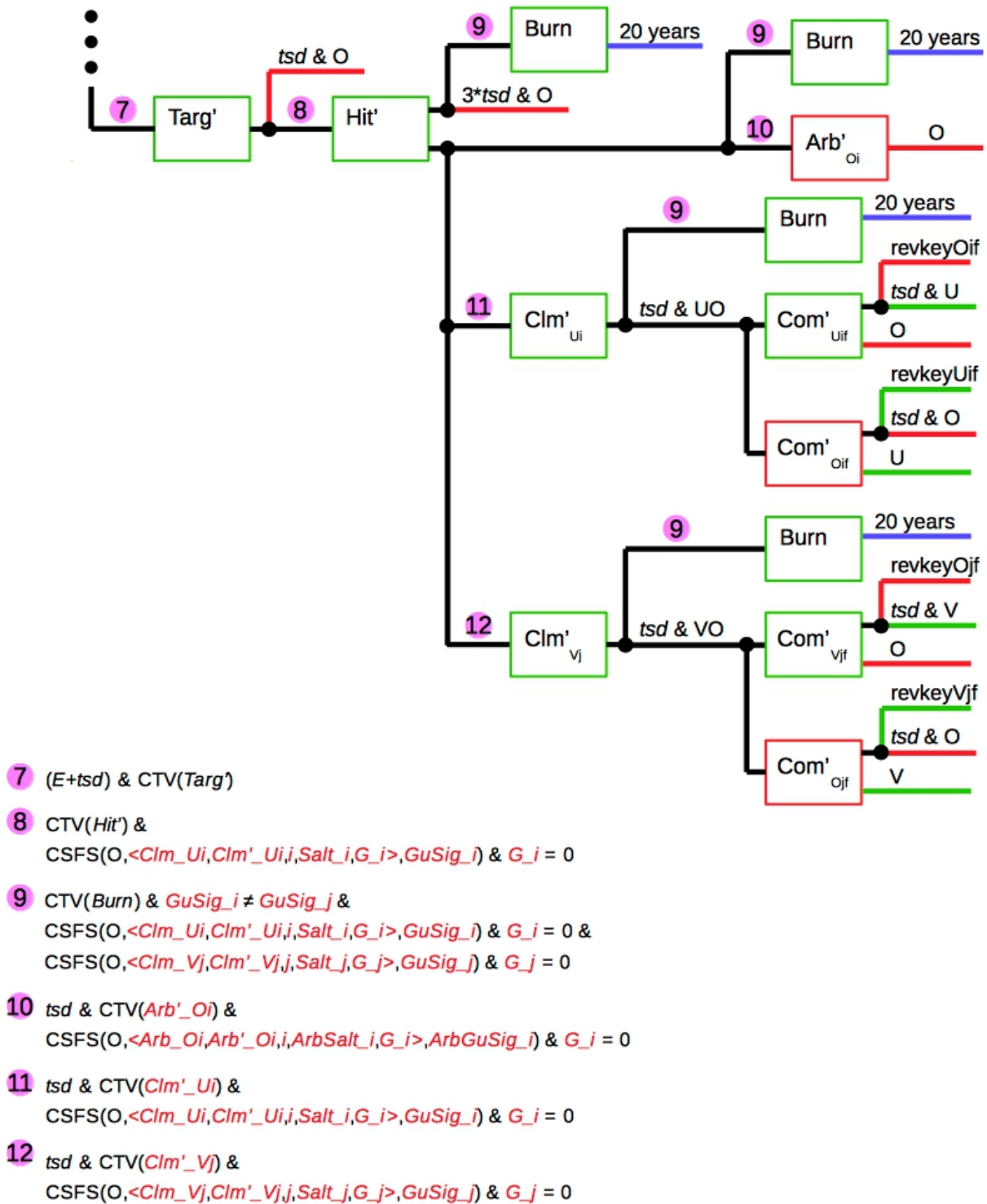


Figure 2. Default resolution of a simplified depot when no operator-selected target is provided.

In Figure 1, O creates a depot by putting a single-output Funding (F) transaction on-chain. After the depot's expiry E and before $E+tsd$, O uses a Target (Targ) transaction to spend F's output. This Target transaction reveals the target value t by providing O's target signature, $TaSig$, for t .

If no user has a channel that hits the target t , O waits tsd and then claims the Target transaction's output.

On the other hand, if there's a user U with a channel i that hits the target t , U puts a Hit transaction on-chain that reveals that channel i is a hit. More precisely, the witness for the Hit transaction includes the target value t , where t is proven to be the target by providing O's target signature, $TaSig$, for t which U obtained when the Target transaction was put on-chain. The witness for the Hit transaction also includes O's guess signature, $GuSig_i$, for the value $\langle Clm_Ui, Clm'_Ui, i, Salt_i, G_i \rangle$ where Clm_Ui and Clm'_Ui are the hashes of Claim transactions that allow U to put a Lightning channel on-chain, i is the index of the channel that's a hit, and $G_i = t$. The signature $GuSig_i$ was given by O to U when U purchased channel i .

The Hit transaction has two outputs, numbered 0 and 1, each of which has exactly half of the Hit transaction's input value⁷.

If U is the owner of the only channel that hits, U puts a Claim transaction (Clm_Ui) on-chain that spends the Hit transaction's output 1. Then, after a delay of tsd , either U or O can put the final Commitment transaction (Com_Uif or Com_Oif) for channel i 's associated Lightning channel on-chain.

On the other hand, if there's more than one hit, users burn both of the Hit transaction's outputs. In particular, if a user has a channel $j \neq i$ that hits, that user will burn both of the Hit transaction's outputs by providing the guess i signature $GuSig_i$ that was revealed in the witness for the Hit transaction and the guess j signature $GuSig_j$ that the user obtained from O when purchasing channel j .

A final possibility is that O puts an arbitrary transaction Arb_Oi on-chain that spends the Hit transaction's output 1, thus grieving U by making U lose the value that Com_Uif would pay to U. However, in doing so O must reveal a guess signature $ArbGuSig_i \neq GuSig_i$, thus allowing U to burn the Hit transaction's output 0. The depot protocol puts a lower bound (*OperatorChannelMin*) on O's balance in a Lightning channel, thus guaranteeing that O's use of an Arb_Oi transaction will result in a net loss to O that's large enough (based on the grieving parameter GrO) to prevent O from putting Arb_Oi on-chain.

In Figure 1, it may not be immediately clear why separate Hit and Claim transactions are used rather than having a single Hit-Claim transaction with two outputs, one matching the Hit transaction's output 0 and the other matching the Claim transaction's output. Note that in Figure 1 the Target's output can only be spent by a Hit transaction that has two equal-valued outputs and the appropriate spending requirements. This is because the Hit transaction must satisfy the $CTV(Hit)$ requirement where the hash

⁷ In order to simplify the presentation, here and throughout the paper the values of outputs will be given ignoring the fees that are required to put the transaction on-chain. In practice, all transaction outputs will have to be reduced slightly in order to accommodate fees. However, the reduction is very slight relative to the value of the output because a single depot aggregates funds from many thousands or millions of Lightning channels.

value *Hit* comes from the Target output's tapleaf script (and is shown in black in the figure). On the other hand, each user has a different Claim transaction that allows that user to put their Lightning channel on-chain, so the tapleaf script for the Hit transaction's output 1 cannot require a CTV with a predefined hash value. Instead, the Hit transaction's output 1 requires that the spending transaction satisfies $CTV(Clm_Ui)$ where *Clm_Ui* is the hash of a transaction that's included in a guess signature *GuSig_i* for a hit (and *Clm_Ui* comes from the witness stack and is shown in red in Figure 1). If a combined Hit-Claim transaction were used to spend the Target transaction's output, different Hit-Claim transactions for different users would have to be supported, so the Hit-Claim transaction could not be required to have a predefined hash value. The Hit-Claim transaction could be required to have a hash for which O created a guess signature *GuSig_i* for a hit. However, that would allow O to create an arbitrary transaction *Arb_i* with a hash *Arb_i* that's included in a guess signature *ArbGuSig_i* for a hit that spends the Target transaction's output without giving users any way to burn the depot's funds. As a result, a combined Hit-Claim transaction would allow O to steal all of the depot's funds.

Figure 2 shows the operation of the protocol when O fails to put a Target transaction on-chain before $E+tsd$. In this case, any user can use the default Target' (Targ') transaction to spend the Funding transaction's output⁸. The default Target' (Targ') transaction's witness does not provide a target value *t*. Instead, the protocol in Figure 2 matches the protocol in Figure 1 except it uses default transactions (indicated with a prime) that require a target with a fixed value of 0.

3.2 Actual Depot Protocol

While the above simplified protocol captures the essence of how channels are created, the actual depot protocol differs in four ways.

First, rather than having only one target *t*, O creates a vector of *h* targets, $\bar{T} = t_1..t_h$ where $0 \leq t_y < P$ for $1 \leq y \leq h$. O's signature for channel *i* includes a selector, *s_i*, $1 \leq s_i \leq h$, such that channel *i* is a hit if it hits target *t_(s_i)* (that is, it hits target *t_y* where $y = s_i$). Having multiple targets allows O to sell more channels, all of which have an independent chance of being a hit.

Second, O's signature for channel *i* covers not only U's guess *G_i* (and its associated salt value) but also a vector of *g* factors $\bar{F}_i = f_{i1}..f_{ig}$ where $0 \leq f_{ix} < P$ for $1 \leq x \leq g$. The factors $\bar{F}_i = f_{i1}..f_{ig}$ are chosen randomly, uniformly and independently by O and given to U with the signature for channel *i*. After the expiry when O selects the targets $\bar{T}$, O also chooses randomly, uniformly and independently a vector of *g* numbers $\bar{R} = r_1..r_g$ where $0 < r_x < P$ for $1 \leq x \leq g$.

A channel *i* is a hit if $\bar{R} * \bar{F}_i + G_i = t_{(s_i)} \pmod{P}$ where $\bar{R} * \bar{F}_i$ denotes the inner product of $\bar{R}$ and $\bar{F}_i$ (that is, $\bar{R} * \bar{F}_i = r_1 * f_{i1} + r_2 * f_{i2} + \dots + r_g * f_{ig}$). The value $(\bar{R} * \bar{F}_i + G_i) \pmod{P}$ is called the *effective guess* for channel *i*. Note that the addition (mod *P*) of the value $\bar{R} * \bar{F}_i$ to the guess *G_i* results in the effective guesses for channels being uniformly and independently distributed, as all possible

⁸ This default Target transaction can be calculated by each user, as it is a deterministic function of the depot's parameters and O's public key.

values of $\bar{R} * \bar{F}_i \bmod P$ are equally likely when U selects G_i . This property will be essential when analyzing O's ability to select a target that differs from the effective guesses of the revoked channels⁹.

Furthermore, the value g is chosen such that, even if U is able to learn the values $\bar{F}_i$ and G_i for each revoked channel i , it's computationally infeasible for U to evaluate the possible sets of values for the effective guesses of those revoked channels¹⁰. This is important as it prevents U from analyzing the revoked channels in order to determine if one of U's active channels has a greater than $1/P$ chance of being a hit (thus allowing U to selectively keep that guess active after the depot's expiry and receive an expected value for the channel that exceeds the channel's cost).

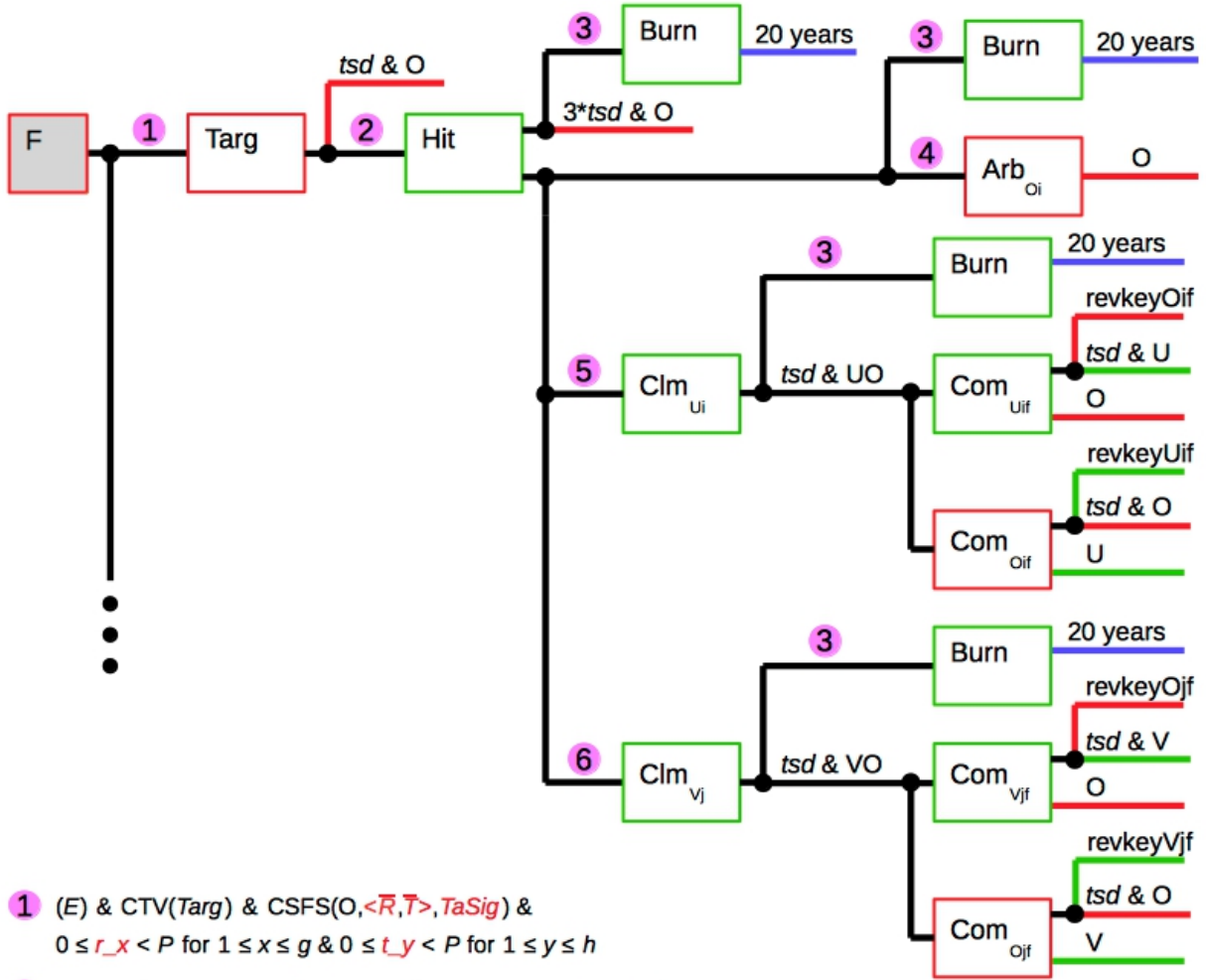
Third, rather than O giving U O's signature for channel i , O gives U an adaptor signature for channel i . This is done to allow U's purchase of channel i to be atomic with O getting a Lightning PTLC payment (from U to O) that pays for the channel (as will be explained below).

Fourth, the salt value that U combines with U's guess for channel i is a joint signature, by U and O, for the value i . U doesn't share the value of this signature with O, but U does give O an adaptor for this signature. This is done to allow U's revocation of channel i to be atomic with O getting a Lightning PTLC receipt for a payment (from O to U) that pays for the revocation (as will be explained below).

The transactions for the full protocol are shown in Figures 3 and 4 below. Figure 3 shows the transactions put on-chain when O provides the target values and Figure 4 shows the transactions put on-chain when O fails to provide the target values within the required time window.

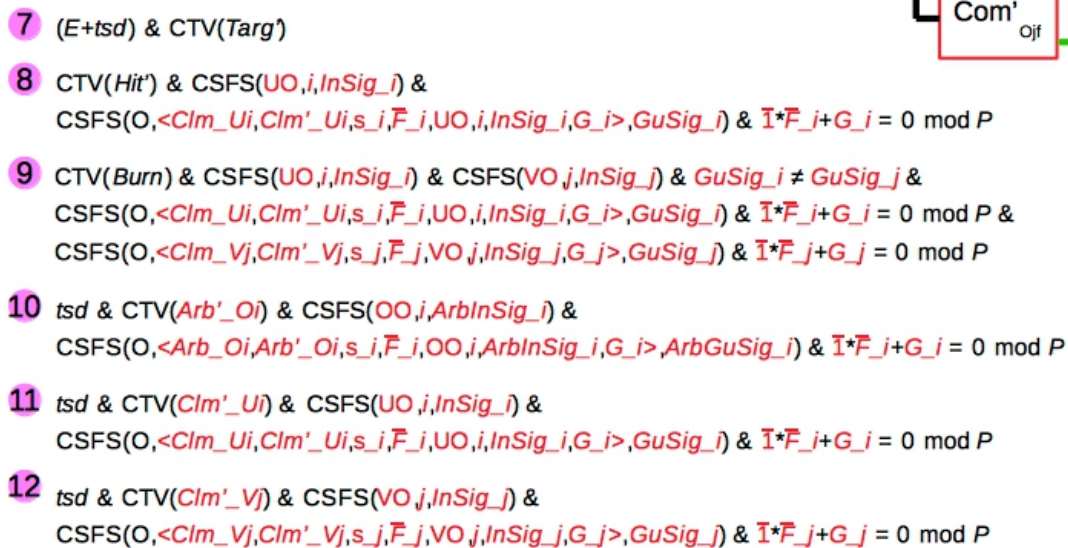
⁹ This analysis is given in Appendix I.

¹⁰ Specifically, each of the g random numbers r_x can take on P different values, so there are P^g possible values of the random vector $\bar{R}$. Given P , the value g is chosen such that $P^g \geq 2^{512}$, thus making the evaluation of the possible values of the effective guesses of the revoked channels computationally infeasible.



- 1 $(E) \& CTV(Targ) \& CSFS(O, \langle \bar{R}, \bar{T} \rangle, TaSig) \& 0 \leq r_x < P \text{ for } 1 \leq x \leq g \& 0 \leq t_y < P \text{ for } 1 \leq y \leq h$
- 2 $CTV(Hit) \& CSFS(O, \langle \bar{R}, \bar{T} \rangle, TaSig) \& CSFS(UO_i, InSig_i) \& CSFS(O, \langle Clm_{Ui}, Clm'_{Ui}, s_i, \bar{F}_j, UO_i, InSig_i, G_i \rangle, GuSig_i) \& \bar{R} * \bar{F}_j + G_i = t(s_i) \text{ mod } P$
- 3 $CTV(Burn) \& CSFS(O, \langle \bar{R}, \bar{T} \rangle, TaSig) \& CSFS(UO_i, InSig_i) \& CSFS(VO_j, InSig_j) \& GuSig_i \neq GuSig_j \& CSFS(O, \langle Clm_{Ui}, Clm'_{Ui}, s_i, \bar{F}_j, UO_i, InSig_i, G_i \rangle, GuSig_i) \& \bar{R} * \bar{F}_j + G_i = t(s_i) \text{ mod } P \& CSFS(O, \langle Clm_{Vj}, Clm'_{Vj}, s_j, \bar{F}_j, VO_j, InSig_j, G_j \rangle, GuSig_j) \& \bar{R} * \bar{F}_j + G_j = t(s_j) \text{ mod } P$
- 4 $tsd \& CTV(Arb_{Oi}) \& CSFS(O, \langle \bar{R}, \bar{T} \rangle, TaSig) \& CSFS(OO_i, ArbInSig_i) \& CSFS(O, \langle Arb_{Oi}, Arb'_{Oi}, s_i, \bar{F}_j, OO_i, ArbInSig_i, G_i \rangle, ArbGuSig_i) \& \bar{R} * \bar{F}_j + G_i = t(s_i) \text{ mod } P$
- 5 $tsd \& CTV(Cl_{m_{Ui}}) \& CSFS(O, \langle \bar{R}, \bar{T} \rangle, TaSig) \& CSFS(UO_i, InSig_i) \& CSFS(O, \langle Clm_{Ui}, Clm'_{Ui}, s_i, \bar{F}_j, UO_i, InSig_i, G_i \rangle, GuSig_i) \& \bar{R} * \bar{F}_j + G_i = t(s_i) \text{ mod } P$
- 6 $tsd \& CTV(Cl_{m_{Vj}}) \& CSFS(O, \langle \bar{R}, \bar{T} \rangle, TaSig) \& CSFS(VO_j, InSig_j) \& CSFS(O, \langle Clm_{Vj}, Clm'_{Vj}, s_j, \bar{F}_j, VO_j, InSig_j, G_j \rangle, GuSig_j) \& \bar{R} * \bar{F}_j + G_j = t(s_j) \text{ mod } P$

Figure 3. Actual protocol for normal resolution of a depot using operator-selected targets and randomization.



Page 14

3.3 Channel Allocation

As was mentioned earlier, the Funding transaction has a single Taproot output where the internal key is the root of a merkle tree containing the parameters of the depot and the root of a merkle-sum tree [Osunto21] that specifies the users of the depot and the allocation of a maximum number of channels to each user. The format of the merkle tree must be standardized so that all users parse it identically.

For each user U , O gives U paths in the merkle tree and the merkle-sum tree that specify the depot's parameters and U 's channel allocation. U evaluates these values to ensure that they meet the requirements of the depot protocol and U 's needs. In addition, U verifies that the Funding transaction's tapleaf scripts are correct given the depot's parameters and O 's public key.

Note that while paths in the merkle tree are given to users in messages, they are never put on-chain. As a result, the lengths of those paths (which are logarithmic in the number of users n) do not affect the worst-case size of the on-chain footprint of a depot (which is constant).

3.4 Creating The Initial Channels And Initial Balance

In order to use a depot to send and receive bitcoin, U must first obtain one or more active channels containing an initial balance¹¹. To do this, U creates one or more off-chain Lightning channels with O and purchases those channels by sending a Lightning payment to O . This Lightning payment can be from another depot or from a custodial account if U doesn't have self-custodial bitcoin, but it must come from U .

To sell channel i , O creates an adaptor signature, with adaptor X , for guess signature $GuSig_i$. Next, U makes a PTLT-based Lightning payment to O where the secret for the payment is X and the value of the payment equals the cost of channel i (and U 's balance in channel i when channel i becomes active). U then uses the receipt for the payment, X , to complete the adaptor signature and acquire channel i from O . An example is shown in Figure 5 below.

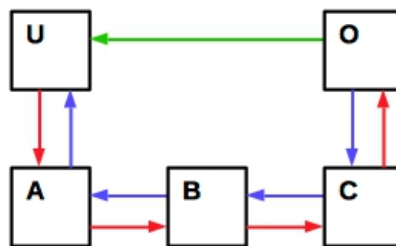


Figure 5. U funding their depot Lightning channel with O .

¹¹ This initial balance gives U "skin in the game", thus enabling U to put funds in a burn output to make griefer-penalized Lightning sends and receives (as will be described later).

In Figure 5, red arrows show the Lightning payment from U to O via A, B and C. Blue arrows show the Lightning payment's PTLC secret X . When U receives the secret X from A, U uses X to complete the guess signature $GuSig_i$, thus granting channel i from O to U as indicated by the green arrow. If desired, U can make a single payment, with secret X , to acquire multiple channels simultaneously by first obtaining adaptor signatures, all with adaptor X , for each channel's signature.

3.5 Revoking The Final Channels And Draining Funds

Before a depot expires, U should revoke all their active channels and drain their funds from the depot. This is done by making a Lightning payment from O to U. This Lightning payment can be to another depot or to a custodial account, but it must be to U¹².

To revoke channel i , O makes a PTLC-based Lightning payment to U where the value of the payment equals U's balance in channel i and the secret for the payment, X , equals the adaptor for the index signature $InSig_i$ that acted as a salt value when U purchased channel i . O then uses the receipt for the payment, X , to complete the adaptor signature. This allows O to determine the salt value that was used, and therefore to calculate U's guess G_i that was combined with the salt value (by grinding through all P possible values, if necessary¹³), thus revoking the active channel from U. An example is shown in Figure 6 below.

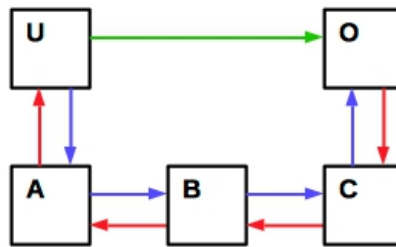


Figure 6. U draining their depot Lightning channels with O.

In Figure 6, red arrows show the Lightning payment from O to U via C, B and A. Blue arrows show the Lightning payment's PTLC secret X . The green arrow shows the revocation of an active channel by U when O gets the receipt, X , for the Lightning payment that completes the adaptor signature.

3.6 Paying On-Chain Fees

Because the Target, Hit and Burn transactions shown in Figure 3 must satisfy a CTV covenant that's set when the depot's Funding transaction is put on-chain, these transactions will pay a fee to the miner that's chosen when the depot is created. The fact that a single depot can support hundreds of thousands or millions of users implies that on-chain fees should be small compared to the value of depot

¹² This is required because the draining of funds eliminates U's "skin in the game", thus preventing U from securing the channel revocation by putting funds in a burn output within the channel being revoked.

¹³ If U follows the protocol, once the payment to U has been made, U also gives O the value G_i , thus saving O from grinding through the calculation of G_i .

transaction outputs. As a result, the Target, Hit and Burn transactions can afford to have conservatively high fees that will ensure they can be put on-chain quickly. However, in the rare case where the anticipated fee is insufficient, it's important to have a way to pay a higher fee if required. One way to do this is to add outputs to the Funding transaction that can be used to fund child-pays-for-parent (CPFP) **[CPFP]** transactions that can be spent by the Targ, Hit and Burn transactions, if needed. Several different CPFP transactions could be available for each Targ, Hit and Burn transaction, where the CPFP transactions differ in the fees paid (with the remainder going to O) and the absolute timelock that's required for putting them on-chain (in order to prevent users from forcing O to pay excessive fees).

The Claim and Commitment transactions are more flexible because they satisfy CTV covenants that are provided by witnesses. However, similar CPFP transactions could be used for the Claim and Commitment transactions in order to avoid having to sign a large number of versions of those transactions that pay different fees.

4 The Off-Chain Lightning Protocol

For each user U, a depot creates unannounced Lightning channels owned by U and O. These Lightning channels can be used to send and receive bitcoin over the Lightning Network, but unlike regular unannounced Lightning channels these channels can't be put on-chain until after the depot's expiry and the ability to put them on-chain is probabilistic. In addition, there are bounds on the amount any one channel pays to O and on U's balance as a function of the number of U's active channels and U's allocation. As a result, depot Lightning channels operate differently from on-chain Lightning channels.

4.1 Scaling Values

As was noted earlier, U has at most a $1/P$ chance of putting each of their Lightning channels on-chain. Therefore, if U sends (or receives) a Lightning payment of X sats, U's channels are updated such that the sum of those channels' payments to U is decreased (or increased) by $X \cdot P$ sats, causing U's balance in the depot to be decreased (or increased) by X sats.

4.2 Resolving Sends And Receives Off-Chain

4.2.1 Use of Burn Outputs

Because a depot channel can't be put on-chain until after the depot's expiry (and only probabilistically even then), it's impossible to use a standard PTLC to resolve Lightning sends and receives in a depot channel. Instead, prior to performing the send or receive, U and O update one or more channels by moving some funds from the outputs that pay U and O to burn outputs¹⁴. Specifically, each party moves a fixed fraction of the payment amount (called *matching funds*) from their outputs to the burn outputs. In addition, U (in the case of a send from U) or O (in the case of a receive by U) moves the payment

¹⁴ A burn output pays to anyone after a 20-year delay, thus providing essentially no expected value to either U or O while rewarding future miners.

amount (scaled by multiplying it by P as described in Section 4.1) from their output to the burn outputs. When the payment is resolved, U and O update the channels by returning the matching funds to their outputs and moving the payment funds from the burn outputs to O's outputs (in the case of a send from U) or U's outputs (in the case of a receive by U).

The key is that during the entire process both U and O have sufficient funds in the burn output to incentivize them to resolve the payment correctly. Specifically, U can grief O by making O lose an expected amount X , but in doing so U must expect to lose at least $GrU * X$. Similarly, O can grief U by making U lose an expected amount X , but in doing so O must expect to lose at least $GrU * X$. Formulas for determining U's and O's contributions to the burn output are given in Appendix F and routines for implementing payments by moving funds to and from burn outputs are given in the program `depot.py` [Law26].

4.2.2 Implementing Burn Outputs

In the current Lightning protocol, old channel states are made unusable via revocation keys. In order to revoke Alice's transactions for channel state i , Alice gives Bob her private key for her per-commitment public key i . Bob creates a linear combination of this private key and his revocation basepoint private key to create his revocation private key i . Knowledge of Bob's revocation private key i can be used to spend all of Alice's outputs from Alice's transactions for state i . As a result, if Alice puts her transactions for state i on-chain, Bob will be able to claim all of the channel's funds¹⁵.

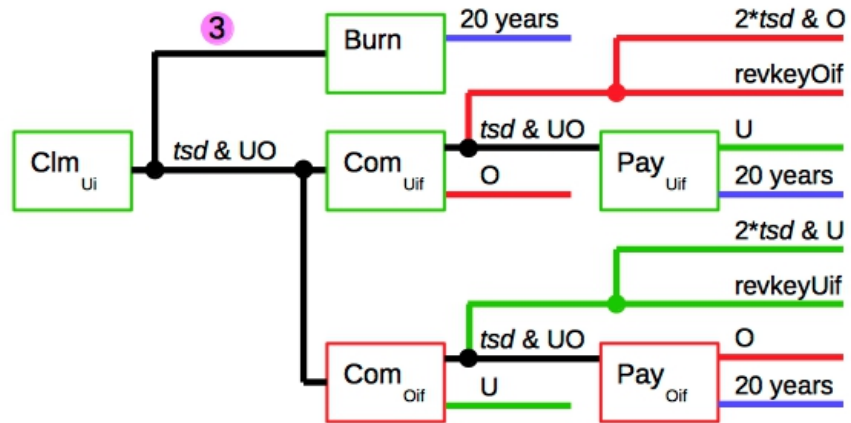
The simplest way to modify the current Lightning protocol to support burn outputs is by adding a burn output to Alice's state i Commitment transaction. However, this approach isn't safe, as Alice could still put an old state i Commitment transaction on-chain, in which case the funds in the burn output for state i would be unavailable to Alice and Bob. Because there's no guaranteed relationship between the size of the burn output in an old state i and Bob's current funds in the channel, there's no guarantee as to the relative sizes of Alice's and Bob's losses if Alice grieves Bob by putting state i on-chain.

This problem could be addressed by allowing the burn output in Alice's state i Commitment transaction to also be spent using revocation private key i . This change assures that Alice can't grief Bob by putting an old state on-chain without Bob gaining all of the funds in the channel (minus fees). Unfortunately, it introduces another problem, as Alice could put the current state on-chain. If Alice does that, neither Alice nor Bob could spend the burn output. However, they have 20 years in which to decide that they would each be better off if they split the burn output, so they may choose to mutually sign a transaction that does so. The problem with this outcome is that it eliminates or reduces Alice's penalty for putting the current state on-chain, so she may choose to do so in anticipation of such an outcome¹⁶.

15 The ideas in this subsection originally appeared in Section 9.1 of [Law24]. They are repeated here for completeness.

16 This may not be a problem in practice, as experience with Lightning when on-chain fees exceed payment amounts shows that users resist such bullying attacks in order to avoid being taken advantage of (see Section 2.2.3). The reason this potential problem is addressed here is that the 20-year time frame for this bullying attack could result in different behavior.

This problem is an example of a bullying attack as discussed in Section 2.2.3, and a solution is to guarantee that all channel updates are performed automatically by wallet software, rather than being under human control. One way to do this is to create a new Payment transaction that has both the current state's *to_self* and burn outputs, but which can only be put on-chain after the given state is proven to not have been revoked. This solution is shown in Figure 7 below.



3 CTV(Burn) & CSFS(O,< $\bar{R}$, $\bar{T}$ >,TaSig) & CSFS(UO,i,InSig_i) & CSFS(VO,j,InSig_j) & GuSig_i $\neq$ GuSig_j & CSFS(O,<CIm_Ui,CIm'_Ui,s_i,F_j,UO,i,InSig_i,G_i>,GuSig_i) & $\bar{R} \cdot \bar{F}_i + G_i = t(s_i) \bmod P$ & CSFS(O,<CIm_Vj,CIm'_Vj,s_j,F_j,VO,j,InSig_j,G_j>,GuSig_j) & $\bar{R} \cdot \bar{F}_j + G_j = t(s_j) \bmod P$

Figure 7. Use of Payment transactions to reduce the likelihood of a bullying attack.

4.2.3 Minimizing Burns

While the use of burn outputs should be sufficient to deter any intentional grieving attacks, there's still a possibility that U and O both try to resolve a payment correctly, but disagree on the resolution. For example, U could believe they provided a payment's secret to O within the time required by the payment's PTLC, but O could believe the revelation came late due to communication latency and clock drift. Such disagreements can be reduced or eliminated by¹⁷:

1. including timestamps in all messages sent between U and O,
2. sending and receiving frequent timestamped "ping" messages to and from independent third parties,
3. having each party maintain a nonvolatile log of all timestamped messages sent and received,
4. using the timestamped messages to synchronize the parties' clocks,
5. sending multiple encrypted copies of PTLC resolution messages using different communication paths via independent third parties, and

¹⁷ A similar discussion is included in Section 3 of [Law24].

6. including a buffer in the PTLC's expiry that's large enough to cover the maximum possible clock skew and message latency.

For example, if U and O are able to synchronize their clocks to within 10 seconds of each other and if they determine that 10 seconds is the maximum latency for communication between them, then U could believe they must send a PTLC's secret at least 20 seconds before O believes the PTLC expires.

If all of the multiple encrypted copies of the PTLC resolution messages (described in item 5 above) are sent but fail to reach the recipient before the PTLC's expiry, it's likely there was either a complete loss of communication by the sender or a failure of the receiver. These cases can be differentiated by looking at the nonvolatile logs for the "ping" messages sent to and from third parties (described in item 2 above).

4.2.4 Migrating Burn Outputs

Both U and O lose funds if a burn output can't be removed, so they're incentivized to use the techniques listed above to agree on the removal of all burn outputs. However, if U and O can't eliminate a burn output, the protocol requires that they migrate the burn output to another depot before the current depot's expiry.

If U and O have funds in a different depot with a later expiry, burn outputs in the current depot with value $X + Y$ sats can be migrated to the other depot as follows:

- in the other depot, U contributes X sats and O contributes Y sats to a new burn output with value $X + Y$, then
- in the current depot, U and O eliminate burn outputs by returning X sats to U and Y sats to O.

The key is that throughout the migration the balance between the parties' burn contributions remains within the depot's GrU and GrO security parameters. Subroutines for increasing and decreasing burn contributions while maintaining the required balance are given in the program `depot.py` [Law26].

4.2.5 Burn Depots

Alternatively, rather than migrating the burn amount to a regular depot operated by O, U and O could choose to migrate the burn amount to a *burn depot* operated by O, where a burn depot is a modified form of depot that's designed solely for holding burn amounts. Specifically, a burn depot is similar to a regular depot except in a burn depot:

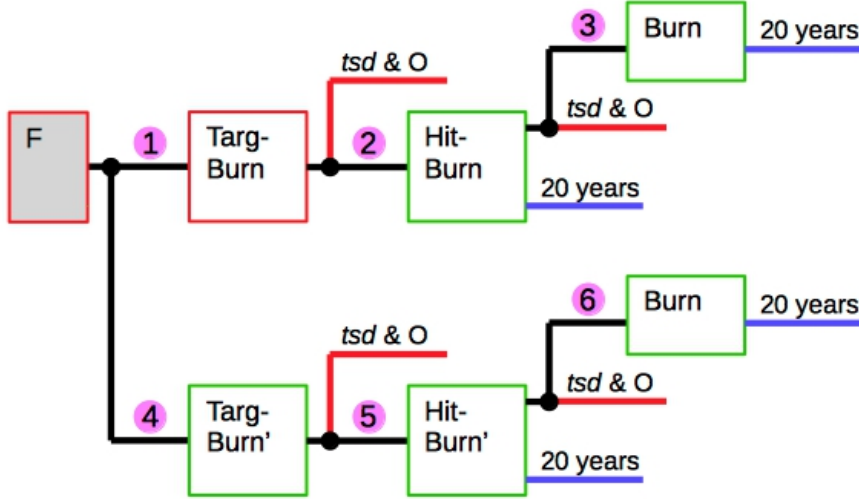
- all funds in the burn depot are either burned or returned to O,
- users can't make Lightning payments to or from the burn depot, and
- if there's exactly one hit in the burn depot after its expiry, half of the funds in the burn depot are burned and the other half are returned to O.

As with a regular depot:

- if there are no hits after the expiry all funds in the burn depot are returned to O, and
- if there are two or more hits after the expiry, all funds in the burn depot are burned.

Due to its limited functionality, U can purchase channels in a burn depot, but U can't drain funds from a burn depot and doesn't revoke channels in a burn depot.

The transactions that implement a burn depot are simpler than those that implement a regular depot, as is shown in Figure 8 below.



- 1 (E) & CTV(Targ-Burn) & CSFS(O, <R, T>, TaSig) & $0 \leq r_x < P$ for $1 \leq x \leq g$ & $0 \leq t_y < P$ for $1 \leq y \leq h$
- 2 CTV(Hit-Burn) & CSFS(O, <R, T>, TaSig) & CSFS(UO, i, InSig_i) & CSFS(O, <CIm_Ui, CIm'_Ui, s_i, F_j, UO, i, InSig_i, G_i>, GuSig_i) & $\bar{R} * F_j + G_i = t(s_i) \mod P$
- 3 CTV(Burn) & CSFS(O, <R, T>, TaSig) & CSFS(UO, i, InSig_i) & CSFS(VO, j, InSig_j) & $GuSig_i \neq GuSig_j$ & CSFS(O, <CIm_Ui, CIm'_Ui, s_i, F_j, UO, i, InSig_i, G_i>, GuSig_i) & $\bar{R} * F_j + G_i = t(s_i) \mod P$ & CSFS(O, <CIm_Vj, CIm'_Vj, s_j, F_j, VO, j, InSig_j, G_j>, GuSig_j) & $\bar{R} * F_j + G_j = t(s_j) \mod P$
- 4 (E+tsd) & CTV(Targ-Burn')
- 5 CTV(Hit-Burn') & CSFS(UO, i, InSig_i) & CSFS(O, <CIm_Ui, CIm'_Ui, s_i, F_j, UO, i, InSig_i, G_i>, GuSig_i) & $\bar{I} * F_j + G_i = 0 \mod P$
- 6 CTV(Burn) & CSFS(UO, i, InSig_i) & CSFS(VO, j, InSig_j) & $GuSig_i \neq GuSig_j$ & CSFS(O, <CIm_Ui, CIm'_Ui, s_i, F_j, UO, i, InSig_i, G_i>, GuSig_i) & $\bar{I} * F_j + G_i = 0 \mod P$ & CSFS(O, <CIm_Vj, CIm'_Vj, s_j, F_j, VO, j, InSig_j, G_j>, GuSig_j) & $\bar{I} * F_j + G_j = 0 \mod P$

Figure 8. Protocol for a burn depot.

Because burn depots don't allow users to put user-specific Lightning channels on-chain, the Hit, Claim and Commitment transactions from regular depots are replaced with a combined Hit-Burn transaction

in a burn depot. If a user has a hit, that user puts a Hit-Burn transaction on-chain that reveals the hit and burns half of the depot's value. If there's a second hit, the other half of the depot's value is also burned.

To eliminate burn outputs in a regular depot having a total value of X sats:

1. U and O each contribute matching funds that are a fixed fraction of X sats to the burn outputs in the regular depot¹⁸, then
2. O gives U channels in a burn depot that are expected to burn at least X sats, then
3. the burn outputs in the regular depot are reduced by moving X sats to O's outputs, and
4. each party's matching funds are returned to them in the regular depot.

A quantitative analysis of burn depots is given in Appendix G.

4.3 Securing Against Failures To Drain

If a regular depot has 2 or more hits, the entire value of the depot is burned. As a result, users could attempt to grief the operator by holding active channels past the depot's expiry. Similarly, the operator could attempt to grief users by preventing them from draining their funds from the depot prior to its expiry. These griefing attacks are prevented by placing a lower bound (*UserChannelMin*) on U's balance in each active channel¹⁹ and an upper bound on U's balance in the depot as a function of the number of U's active Lightning channels and U's channel allocation. These bounds guarantee that if one or more users fail to drain their channels in an attempt to grief O and if O's expected loss is X , then the users performing the griefing attack will expect to lose at least $GrU * X$. Similarly, if O fails to allow one or more users to drain their channels in an attempt to grief the users and if the users' expected loss is X , then O will expect to lose at least $GrO * X$. Formulas for lower and upper bounds on U's balance in any active channel and the upper bound on U's overall depot balance are given in Appendix B and are analyzed in Appendices C, D and E.

4.4 Per-Channel Capacities

If U has a balance Bu spread across Au active channels, there could be multiple ways in which U's balance could be partitioned among those channels. The management of U's active channels is simplified if U's balance always follows a single canonical partitioning across channels. In particular, each active channel is assigned a *capacity* and U's balance that exceeds U's per-channel minimum balances is assigned to U's oldest active channel until it reaches its capacity, then to U's second oldest capacity until it reaches its capacity, etc. The per-channel capacities are assigned so that this assignment

18 The matching funds are the same as those required to make a Lightning send from U with value X and are defined in Appendix F.

19 In addition, whenever U has no burn contribution, U's total balance must be slightly larger than the sum of the per-channel *UserChannelMin* values. This requirement guarantees that U will have sufficient funds to create matching burn contributions whenever U needs to revoke a channel. See Table 6 in Section 5.3.2 for an example.

of U's balance to channels satisfies the lower and upper bounds on U's balance in the depot described in Section 4.3. Formulas for per-channel capacities are given in Appendix B.

4.5 Routing Lightning Payments

Theoretically, depot Lightning channels could be used for routing Lightning payments in addition to sending and receiving them. However, there are several reasons why depots are less suitable than timeout trees [Law21][Law23b] for creating channels that route payments:

- Routing channels need to support very high numbers of payments. Depot Lightning payments can require channels to be purchased or revoked and each depot has a bounded number of channels. In contrast, channels in timeout trees can route an unbounded number of payments.
- It's likely that a relatively small number of routing channels, each having a relatively large balance, can be used to support a very large number of depot channels that send and receive payments. As a result, depots' extreme scalability and support for tiny balances are not required for routing channels.

While any Lightning protocol can be used within routing channels, supporting very high payment rates (e.g., tens or hundreds of thousands of payments per second) is likely to require use of the Off-chain Payment Resolution (OPR) protocol [Law24] within routing channels, as the OPR protocol eliminates the need to go on-chain to resolve payments. In addition, the OPR protocol is attractive because it can provide guaranteed payment resolution within seconds (as opposed to hours or days).

5 Example

5.1 Parameters

Assume a depot has the following parameters:

- $GrO = 0.05$ (so if O causes users to lose an expected value of x , O must expect to lose at least $0.05 \cdot x$ of O's own funds)
- $GrU = 0.30$ (so if users cause O to lose an expected value of x , users must expect to lose at least $0.30 \cdot x$ of their own funds)
- $balance_range = 1.5$ (so given any fixed number of active channels, the maximum possible user's balance is at least 50% larger than the minimum possible user's balance),
- $n = 1,000,000$ users,
- $MaxA = 10,000,000$ maximum active channels (so an average user can be allocated 10 channels),

- $MaxC = 1,000,000,000$ maximum cumulative number of channels created over the lifetime of the depot (so an average user can acquire and revoke up to 1,000 channels during the lifetime of the depot),
- $supported_user_balances = 100,000,000$ sats (so users can have balances that total at least 100,000,000 sats),
- $to_self_delay = 10,000$ blocks (about 2.5 months) (that is, users that attempt to go on-chain must monitor the blockchain at least once every 2.5 months), and
- E = the depot's expiry is 100,000 blocks (about 2 years) after the depot is put on-chain.

Given the above parameters, the following parameters can be calculated²⁰:

- $D = 107,718,596$ sats (so the depot is funded by the operator with 107,718,596 sats),
- $P = 1,864,361$ (so each channel has a $1/1,864,361$ chance of being a hit),
- $OperatorChannelMin = 2,564,729/P = 1.376$ sats (so every active channel must pay O at least 2,564,729 sats and O's balance in every active channel must be at least 1.376 sats),
- $UserChannelMin = 12,429,069/P = 6.667$ sats (so every active channel must pay a user at least 12,429,069 sats and each user's balance in every active channel must be at least 6.667 sats),
- $MaxH = 5.364$ maximum expected number of hits (so there are 5.364 expected hits if all $MaxA$ channels are active),
- $g = 25$ (so the vectors $\bar{R}$ and $\bar{F}_i$ each have 25 components),
- $h = 54$ targets (so each channel is assigned a target t_y where $1 \leq y \leq 54$), and
- $max_utilization = 0.9283$ (so up to 92.83% of the depot's value can be sold to users).

5.2 Single Depot Analysis²¹

5.2.1 Per-User Balances And Per-Channel Capacities

Table 1 shows a typical²² user U's minimum and maximum balance and per-channel capacity as a function of the number of U's active channels.

²⁰ Routines for calculating these parameters are given in the program depot.py [Law26].

²¹ The program depot.py [Law26] can be used to calculate the values presented in this section for an arbitrary depot.

²² In particular, Table 1 shows minimum and maximum balances for U assuming U has an allocation of 10 active channels (which is the depot's average allocation of channels per user). Other users could have different allocations, in which case they would have different minimum and maximum balances and per-channel capacities as a function of their number of active channels. Users with an allocation of 10 active channels have a *min_nonzero_channels* value of 1, meaning that they only require 1 active channel in order to support a nonzero balance in the depot (which must be at least 8.799 sats, as this is the minimum value for a user with 1 active channel and 10 allocated channels and no contribution to burn outputs).

<i>Au</i>	<i>min_balance</i>	<i>min_balance_no_burn</i>	<i>max_balance</i>	<i>min_per_channel</i>	<i>max_per_channel</i>
1	6.667	8.799	25.065	6.667	25.065
2	13.333	15.465	45.372	6.667	20.306
3	20.000	22.132	61.303	6.667	15.932
4	26.667	28.799	73.422	6.667	12.119
5	33.333	35.465	82.378	6.667	8.955
6	40.000	42.132	88.824	6.667	6.667
7	46.667	48.799	93.356	6.667	6.667
8	53.333	55.465	96.479	6.667	6.667
9	60.000	62.132	98.592	6.667	6.667
10	66.667	68.799	100.000	6.667	6.667

Table 1. User U's minimum and maximum balances (in sats) in the depot and in each channel as a function of the number of U's active channels (*Au*), assuming U has an allocation of 10 channels.

For example, if U has 1 active channel, then U's balance must be at least 6.667 sats (or 8.799 sats if U has no burn contribution) and 25.065 sats. In contrast, if U has 5 active channels, then U can have a balance in the range of 33.333 sats (or 35.465 sats if U has no burn contribution) to 82.378 sats and U's fifth active channel (U's youngest one) must have a balance of 6.667 sats to 8.955 sats. If all 10 of U's allocated channels are active, U can have a balance in the range of 66.667 sats to 100.000 sats and each of U's 10 channels must have a balance within the range given in the last two columns.

5.2.2 Single Depot Analysis

Over the lifetime of the depot, users can acquire (and then revoke) up to 1,000,000,000 channels with a total value of at most 25,065,000,000 sats (which is 232 times the total value of the depot).

Furthermore, over the lifetime of the depot users can send and receive an unbounded amount of value by varying their payouts from their Lightning channels without acquiring or revoking channels (provided the user's and operator's balances throughout the processing of the payment are within the required bounds for the current number of active channels).

If the operator and all users follow the protocol, all of the users' funds will be drained from the depot prior to its expiry. However, it's likely that at least a few users will fail to drain their funds, so it's useful to consider how those users will impact the operator.

For example, assume all users who don't drain have their full allocation of 10 active channels and their maximum of 100 sats in the depot. If 1% of users fail to drain but remain active and attempt to put their channels on-chain (or burn the depot's outputs if there are multiple users with hits), the operator will expect to lose 97,298 sats, which is 0.09% of the depot's value. On the other hand, if 1/10 as many users (0.1% of the total) fail to drain and are **not** active after the expiry, those users will give to the

operator 100,000 sats, which more than covers the operator's expected loss from users who don't drain.

As a more likely example, assume all users who don't drain have half (5) their full allocation of 10 active channels and half (50 sats) of their maximum of 100 sats in the depot. In this case, if 1% of users fail to drain but remain active and attempt to put their channel on-chain (or burn the depot's outputs if there are multiple users with hits), the operator will expect to lose 24,821 sats, which is 0.023% of the depot's value. If even 1/20 as many users (0.05% of the total) fail to drain and are **not** active after the expiry, those users will give to the operator 25,000 sats, which again more than covers the operator's expected loss from users who don't drain.

In summary, it appears likely that users who don't drain will provide an expected net profit to the operator.

5.3 Funding, Operating And Draining Channels

This section gives an example of how an arbitrary user U with an allocation of 10 channels can fund channels, send and receive payments, and drain channels in the depot²³.

5.3.1 Funding The Channels

Assume U funds their channels in this depot by sending 55 sats to O from channels U owns in a different depot. In order to hold 55 sats D will require 3 active channels (as Table 1 shows that 2 active channels can support a maximum of 45.372 sats, while 3 active channels can support a maximum of 61.303 sats). Therefore, U purchases 3 channels from O and pays O 55 sats for those channels, where the first channel holds 25.065 sats for U²⁴, the second channel holds 20.306 sats for U, and the third channel holds the remaining 9.628 sats for U.

In order to sell these 3 channels to U, O gives U adaptor signatures, with adaptor X , for guess signatures $GuSig_i$, $GuSig_j$ and $GuSig_k$. U's 55-sat Lightning payment for these 3 channels uses PTLCS with secret X , so when U obtains the receipt for the payment U can create the full guess signatures $GuSig_i$, $GuSig_j$ and $ChSig_k$ (as shown in Figure 5)²⁵.

The status of U's channels before (Step 0) and after (Step 1) funding channels 1, 2 and 3 is shown in Table 2. This table and the following tables show the number of U's active channels (A_u), the balances in each channel and the total balance across all of U's channels. Each channel shows U's balance in **light green**, U's burn contribution in **dark cyan**, the channel's total burn contributions in **blue**, O's burn contribution in **magenta**, and O's balance in **red**.

23 The program depot.py [Law26] produced all of the updates presented in this example and can be used to determine how to perform an arbitrary sequence of sends and receives for an arbitrary depot.

24 That is, the channel pays U $25.065 * P = 46,730,871$ sats if it's put on-chain.

25 Throughout this analysis, Lightning fees are omitted in order to simplify the presentation.

Step	Notes	Au	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Total
0	Initial state	0						
1	Atomic grant of channels 1 - 3 with 55-sat Lightning payment	3	25.065 0.000 0.000 0.000 3.824	20.306 0.000 0.000 0.000 8.582	9.628 0.000 0.000 0.000 19.261			55.000 0.000 0.000 0.000

Table 2. Example of user U funding U's channels with 55 sats.

5.3.2 Performing Sends And Receives

5-sat Receive

Next, assume U wants to receive a 5-sat Lightning payment from another user V. This will create a balance of 60 sats for U, which is less than the maximum of 61.303 sats that U can have with 3 active channels, so U doesn't have to obtain any more channels in order to receive this payment. As a result, U and O fund burn outputs with sufficient amounts to penalize either party if they fail to resolve the payment correctly. In particular, U contributes 1.599 sats of matching funds to the burn output in channel 3 and O contributes 0.330 sats of matching funds and 5 sats of payment funds to the same burn output, for a total of 6.929 sats in the burn output (Step 2 in Table 3). Next, U gives O the secret for the 5-sat Lightning payment and O uses the secret to obtain payment from O's upstream partner. O's reception of the 5-sat Lightning payment logically moves 5 sats from O's burn contribution to U's burn contribution (Step 3)²⁶. Next, U and O update channel 3 by moving their burn contributions to their respective outputs (Step 4).

Step	Notes	Au	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Total
2	Burn payment and matching funds for 5-sat receive	3	25.065 0.000 0.000 0.000 3.824	20.306 0.000 0.000 0.000 8.582	8.029 1.599 6.929 5.330 13.931			53.401 1.599 6.929 5.330
3	Receive 5 sats	3	25.065 0.000 0.000 0.000 3.824	20.306 0.000 0.000 0.000 8.582	8.029 6.599 6.929 0.330 13.931			53.401 1.599 6.929 5.330
4	Move funds from burn outputs for 5-sat receive	3	25.065 0.000 0.000 0.000 3.824	20.306 0.000 0.000 0.000 8.582	14.628 0.000 0.000 0.000 14.261			60.000 0.000 0.000 0.000

Table 3. Example of user U receiving a Lightning payment of 5 sats.

²⁶ The burn output is unchanged, but the 5 sats contributed by O for the payment are now logically viewed as U's contribution, as O was reimbursed for O's 5-sat contribution when O received the 5-sat Lightning payment.

10-sat Send

Next, assume U wants to make a 10-sat Lightning payment to another user W. In order to implement the 10-sat send, U will have to burn 10 sats for the payment and 3.198 sats of matching funds, thus creating a low balance during the payment of 46.802 sats, which is more than the minimum of 20.000 sats that U must have with 3 active channels. In addition, after performing the 10-sat send, U will have a balance of 50 sats, which is more than the 22.132 sats required when U has no burn contribution. As a result, U doesn't have to revoke any channels in order to make this payment. First, U contributes 3.198 sats of matching funds plus 10 sats of payment funds to the burn output in channels 2 and 3 and O contributes 0.660 sats of matching funds to the same burn outputs, for a total of 13.858 sats in the burn outputs (Step 5 and 6). When O gives U the secret for the 10-sat Lightning payment, U's reception of the secret logically moves 10 sats from O's burn contribution to U's burn contribution (Step 7). Next, U and O update channel 3 by moving their burn contributions to their respective outputs (Steps 8 and 9).

Step	Notes	Au	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Total
5	Burn payment and matching funds in channel 2 for 10-sat send	3	25.065 0.000 0.000 0.000 3.824	15.070 5.236 5.566 0.330 8.252	14.628 0.000 0.000 0.000 14.261			54.764 5.236 5.566 0.330
6	Burn payment and matching funds in channel 3 for 10-sat send	3	25.065 0.000 0.000 0.000 3.824	15.070 5.236 5.566 0.330 8.252	6.667 7.962 8.291 0.330 13.931			46.802 13.198 13.858 0.660
7	Send 10 sats	3	25.065 0.000 0.000 0.000 3.824	15.070 3.198 5.566 2.368 8.252	6.667 0.000 8.291 8.291 13.931			46.802 3.198 13.858 10.660
8	Move funds from burn outputs in channel 3 for 10-sat send	3	25.065 0.000 0.000 0.000 3.824	15.070 3.198 5.566 2.368 8.252	6.667 0.000 0.000 0.000 22.222			46.802 3.198 5.566 2.368
9	Move funds from burn outputs in channel 2 for 10-sat send	3	25.065 0.000 0.000 0.000 3.824	18.268 0.000 0.000 0.000 10.621	6.667 0.000 0.000 0.000 22.222			50.000 0.000 0.000 0.000

Table 4. Example of user U sending a Lightning payment of 10 sats.

30-sat Receive

Now, assume U wants to receive a 30-sat Lightning payment from another user Z. This will create a balance of 80 sats for U, which is greater than the maximum of 61.303 sats that U can have with 3 active channels. As a result, O first grants U 2 additional active channels, each of which holds the minimum user per-channel balance of 6.667 sats for U, thus giving U 5 active channels, which is compatible with U's low balance of 40.406 sats during the payment (as U will have to contribute 9.594 sats of matching funds) and U's after-payment balance of 80 sats. In order to receive a grant of channel 4 with a user balance of 6.667 sats, U burns 6.667 sats plus 2.132 sats of matching funds and O burns 0.440 sats of matching funds (Step 10). When U receives the new channel with 6.667 sats, 6.667 sats are logically moved from being U's burn contribution to being O's burn contribution (Step 11). Then, the funds in the burn output are returned to U and O (step 12). A similar process is performed for granting channel 5 to U (Steps 13 through 17).

Once U has obtained 5 channels, U and O burn payment and matching funds in channels 2 through 5 (Steps 18 through 21). When U gives O the secret for the 30-sat Lightning payment, O uses the secret to obtain payment from O's upstream partner. O's reception of the 30-sat Lightning payment logically moves 30 sats from O's burn contribution to U's burn contribution (Step 22). Next, the funds in the burn outputs are returned to U and O (Steps 23 through 26).

Step	Notes	Au	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Total
10	Burn funds in channel 2 for grant of channel 4 for 30-sat receive	3	25.065 0.000 0.000 0.000 3.824	9.469 8.799 9.239 0.440 10.181	6.667 0.000 0.000 0.000 22.222			41.201 8.799 9.239 0.440
11	Grant channel 4 for 30-sat receive	4	25.065 0.000 0.000 0.000 3.824	9.469 2.132 9.239 7.107 10.181	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222		47.868 2.132 9.239 7.107
12	Move funds from burn outputs in channel 2 for channel 4 grant	4	25.065 0.000 0.000 0.000 3.824	11.601 0.000 0.000 0.000 17.288	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222		50.000 0.000 0.000 0.000
13	Burn funds in channel 1 for grant of channel 5 for 30-sat receive	4	21.201 3.864 4.084 0.220 3.604	11.601 0.000 0.000 0.000 17.288	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222		46.136 3.864 4.084 0.220

Step	Notes	Au	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Total
14	Burn funds in channel 2 for grant of channel 5 for 30-sat receive	4	21.201 3.864 4.084 0.220 3.604	6.667 4.935 5.155 0.220 17.068	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222		41.201 8.799 9.239 0.440
15	Grant channel 5 for 30-sat receive	5	21.201 2.132 4.084 1.952 3.604	6.667 0.000 5.155 5.155 17.068	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222	47.868 2.132 9.239 7.107
16	Move funds from burn outputs in channel 2 for channel 5 grant	5	21.201 2.132 4.084 1.952 3.604	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222	47.868 2.132 4.084 1.952
17	Move funds from burn outputs in channel 1 for channel 5 grant	5	23.333 0.000 0.000 0.000 5.556	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222	50.000 0.000 0.000 0.000
18	Burn payment and matching funds in channel 1 for 30-sat receive	5	13.739 9.594 11.821 2.227 3.329	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222	40.406 9.594 11.821 2.227
19	Burn payment and matching funds in channel 2 for 30-sat receive	5	13.739 9.594 11.821 2.227 3.329	6.667 0.000 14.135 14.135 8.088	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222	40.406 9.594 25.956 16.362
20	Burn payment and matching funds in channel 3 for 30-sat receive	5	13.739 9.594 11.821 2.227 3.329	6.667 0.000 14.135 14.135 8.088	6.667 0.000 9.760 9.760 12.462	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222	40.406 9.594 35.715 26.122
21	Burn payment and matching funds in channel 4 for 30-sat receive	5	13.739 9.594 11.821 2.227 3.329	6.667 0.000 14.135 14.135 8.088	6.667 0.000 9.760 9.760 12.462	6.667 0.000 5.858 5.858 16.364	6.667 0.000 0.000 0.000 22.222	40.406 9.594 41.574 31.980
22	Receive 30 sats	5	13.739 11.326 11.821 0.495 3.329	6.667 13.640 14.135 0.495 8.088	6.667 9.265 9.760 0.495 12.462	6.667 5.363 5.858 0.495 16.364	6.667 0.000 0.000 0.000 22.222	40.406 39.594 41.574 1.980

Step	Notes	Au	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Total
23	Move funds in channel 1 from burn outputs for 30-sat receive	5	25.065	6.667	6.667	6.667	6.667	51.732
			0.000	13.640	9.265	5.363	0.000	28.268
			0.000	14.135	9.760	5.858	0.000	29.753
			0.000	0.495	0.495	0.495	0.000	1.485
			3.824	8.088	12.462	16.364	22.222	
24	Move funds in channel 2 from burn outputs for 30-sat receive	5	25.065	20.306	6.667	6.667	6.667	65.372
			0.000	0.000	9.265	5.363	0.000	14.628
			0.000	0.000	9.760	5.858	0.000	15.618
			0.000	0.000	0.495	0.495	0.000	0.990
			3.824	8.582	12.462	16.364	22.222	
25	Move funds in channel 3 from burn outputs for 30-sat receive	5	25.065	20.306	15.932	6.667	6.667	74.637
			0.000	0.000	0.000	5.363	0.000	5.363
			0.000	0.000	0.000	5.858	0.000	5.858
			0.000	0.000	0.000	0.495	0.000	0.495
			3.824	8.582	12.957	16.364	22.222	
26	Move funds in channel 4 from burn outputs for 30-sat receive	5	25.065	20.306	15.932	12.030	6.667	80.000
			0.000	0.000	0.000	0.000	0.000	0.000
			0.000	0.000	0.000	0.000	0.000	0.000
			0.000	0.000	0.000	0.000	0.000	0.000
			3.824	8.582	12.957	16.859	22.222	

Table 4. Example of user U receiving a Lightning payment of 30 sats.

25-sat Send

Next, assume U wants to make a 25-sat Lightning payment to another user S. In order to implement this 25-sat send, U will have to burn 25 sats for the payment and 7.995 sats in matching funds, yielding a low balance during the payment of 47.005 sats, which is greater than the minimum of 33.333 sats that U must have with 5 active channels, and U's final balance of 55 sats is greater than the 35.465 sats required without a burn contribution, so U doesn't have to revoke any channels in order to make this payment. As a result, U contributes payment and matching funds to the burn outputs in channels 1 through 4 and O contributes matching funds to the burn outputs in those channels (Steps 27 through 32). In steps 28 and 29, U's contribution to the burn outputs in channels 2 and 3 is limited, as U's full contribution to those burn outputs would cause O's total burn contribution to be too low relative to U's total burn contribution, given the security parameter $GrO = 0.05$. As a result, U's remaining burn contributions in those channels is delayed until Steps 31 and 32, when O has made sufficient burn contributions. When O gives U the secret for the 25-sat Lightning payment, U's reception of the secret logically moves 25 sats from O's burn contribution to U's burn contribution (Step 33). Next, U and O update channels 1 through 4 by moving their burn contributions to their respective outputs (Steps 34 through 37).

Step	Notes	Au	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Total
27	Burn payment and matching funds in channel 1 for 25-sat send	5	20.338 4.727 5.139 0.412 3.411	20.306 0.000 0.000 0.000 8.582	15.932 0.000 0.000 0.000 12.957	12.030 0.000 0.000 0.000 16.859	6.667 0.000 0.000 0.000 22.222	75.273 4.727 5.139 0.412
28	Burn some payment and matching funds in channel 2 for 25-sat send	5	20.338 4.727 5.139 0.412 3.411	8.536 11.771 12.183 0.412 8.170	15.932 0.000 0.000 0.000 12.957	12.030 0.000 0.000 0.000 16.859	6.667 0.000 0.000 0.000 22.222	63.503 16.497 17.322 0.825
29	Burn some payment and matching funds in channel 3 for 25-sat send	5	20.338 4.727 5.139 0.412 3.411	8.536 11.771 12.183 0.412 8.170	7.683 8.249 8.661 0.412 12.545	12.030 0.000 0.000 0.000 16.859	6.667 0.000 0.000 0.000 22.222	55.254 24.746 25.984 1.237
30	Burn payment and matching funds in channel 4 for 25-sat send	5	20.338 4.727 5.139 0.412 3.411	8.536 11.771 12.183 0.412 8.170	7.683 8.249 8.661 0.412 12.545	6.667 5.363 5.776 0.412 16.447	6.667 0.000 0.000 0.000 22.222	49.891 30.109 31.759 1.650
31	Burn remaining payment and matching funds in channel 2 for 25-sat send	5	20.338 4.727 5.139 0.412 3.411	6.667 13.640 14.052 0.412 8.170	7.683 8.249 8.661 0.412 12.545	6.667 5.363 5.776 0.412 16.447	6.667 0.000 0.000 0.000 22.222	48.021 31.979 33.628 1.650
32	Burn remaining payment and matching funds in channel 3 for 25-sat send	5	20.338 4.727 5.139 0.412 3.411	6.667 13.640 14.052 0.412 8.170	6.667 9.265 9.677 0.412 12.545	6.667 5.363 5.776 0.412 16.447	6.667 0.000 0.000 0.000 22.222	47.005 32.995 34.645 1.650
33	Receive 25 sats	5	20.338 4.727 5.139 0.412 3.411	6.667 3.268 14.052 10.784 8.170	6.667 0.000 9.677 9.677 12.545	6.667 0.000 5.776 5.776 16.447	6.667 0.000 0.000 0.000 22.222	47.005 7.995 34.645 26.650
34	Move funds in channel 3 from burn outputs for 25-sat receive	5	20.338 4.727 5.139 0.412 3.411	6.667 3.268 14.052 10.784 8.170	6.667 0.000 0.000 0.000 22.222	6.667 0.000 5.776 5.776 16.447	6.667 0.000 0.000 0.000 22.222	47.005 7.995 24.967 16.972
35	Move funds in channel 4 from burn outputs for 25-sat receive	5	20.338 4.727 5.139 0.412 3.411	6.667 3.268 14.052 10.784 8.170	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222	47.005 7.995 19.192 11.197

Step	Notes	Au	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Total
36	Move funds in channel 1 from burn outputs for 25-sat receive	5	25.065	6.667	6.667	6.667	6.667	51.732
			0.000	3.268	0.000	0.000	0.000	3.268
			0.000	14.052	0.000	0.000	0.000	14.052
			0.000	10.784	0.000	0.000	0.000	10.784
			3.824	8.170	22.222	22.222	22.222	
37	Move funds in channel 2 from burn outputs for 25-sat receive	5	25.065	9.935	6.667	6.667	6.667	55.000
			0.000	0.000	0.000	0.000	0.000	0.000
			0.000	0.000	0.000	0.000	0.000	0.000
			0.000	0.000	0.000	0.000	0.000	0.000
			3.824	18.954	22.222	22.222	22.222	

Table 5. Example of user U sending a Lightning payment of 25 sats.

30-sat Send

Next, assume U wants to send 30 sats to another user T. If this 30-sat send were implemented as a single Lightning payment, U would have to burn 30 sats for the payment plus 9.594 sats of matching funds, leaving U with a low balance of 15.406 sats during the 30-sat send. In order to have a balance of 15.406 sats, the user can have no more than 2 active channels (see Table 1). However, a user with at most 2 active channels can have a balance of at most 45.372 sats (see Table 1), which is less than the user's current balance of 55 sats. As a result, the 30-sat send can't be implemented with a single Lightning send (as there is no single number of active channels that the user could have when performing that send).

Instead, the desired 30-sat send is performed as two separate Lightning sends, the first for 26.519 sats (Steps 38 through 54) and the second for 3.481 sats (Steps 55 through 57).

Step	Notes	Au	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Total
38	Burn funds in channel 2 for revocation of channel 4 for 26.519-sat send	5	25.065	7.803	6.667	6.667	6.667	52.868
			0.000	2.132	0.000	0.000	0.000	2.132
			0.000	9.239	0.000	0.000	0.000	9.239
			0.000	7.107	0.000	0.000	0.000	7.107
			3.824	11.848	22.222	22.222	22.222	
39	Revoke channel 5 for 26.519-sat send	4	25.065	7.803	6.667	6.667		46.201
			0.000	8.799	0.000	0.000		8.799
			0.000	9.239	0.000	0.000		9.239
			0.000	0.440	0.000	0.000		0.440
			3.824	11.848	22.222	22.222		
40	Move funds in channel 2 from burn outputs for channel 5 revocation	4	25.065	16.601	6.667	6.667		55.000
			0.000	0.000	0.000	0.000		0.000
			0.000	0.000	0.000	0.000		0.000
			0.000	0.000	0.000	0.000		0.000
			3.824	12.288	22.222	22.222		

Step	Notes	Au	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Total
41	Burn funds in channel 2 for revocation of channel 4 for 26.519-sat send	4	25.065 0.000 0.000 0.000 3.824	14.469 2.132 6.057 3.925 8.362	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222		52.868 2.132 6.057 3.925
42	Burn funds in channel 3 for revocation of channel 4 for 26.519-sat send	4	25.065 0.000 0.000 0.000 3.824	14.469 2.132 6.057 3.925 8.362	6.667 0.000 3.181 3.181 19.041	6.667 0.000 0.000 0.000 22.222		52.868 2.132 9.239 7.107
43	Revoke channel 4 for 26.519-sat send	3	25.065 0.000 0.000 0.000 3.824	14.469 5.837 6.057 0.220 8.362	6.667 2.962 3.181 0.220 19.041			46.201 8.799 9.239 0.440
44	Move funds in channel 2 from burn outputs for channel 4 revocation	3	25.065 0.000 0.000 0.000 3.824	20.306 0.000 0.000 0.000 8.582	6.667 2.962 3.181 0.220 19.041			52.038 2.962 3.181 0.220
45	Move funds in channel 3 from burn outputs for channel 4 revocation	3	25.065 0.000 0.000 0.000 3.824	20.306 0.000 0.000 0.000 8.582	9.628 0.000 0.000 0.000 19.261			55.000 0.000 0.000 0.000
46	Burn some payment and matching funds in channel 1 for 26.519-sat send	3	13.399 11.667 12.250 0.583 3.240	20.306 0.000 0.000 0.000 8.582	9.628 0.000 0.000 0.000 19.261			43.333 11.667 12.250 0.583
47	Burn some payment and matching funds in channel 2 for 26.519-sat send	3	13.399 11.667 12.250 0.583 3.240	8.640 11.667 12.250 0.583 7.999	9.628 0.000 0.000 0.000 19.261			31.667 23.333 24.500 1.167
48	Burn payment and matching funds in channel 3 for 26.519-sat send	3	13.399 11.667 12.250 0.583 3.240	8.640 11.667 12.250 0.583 7.999	6.667 2.962 3.545 0.583 18.677			28.705 26.295 28.045 1.750
49	Burn remaining payment and matching funds in channel 1 for 26.519-sat send	3	6.667 18.399 18.982 0.583 3.240	8.640 11.667 12.250 0.583 7.999	6.667 2.962 3.545 0.583 18.677			21.973 33.027 34.777 1.750

Step	Notes	Au	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Total
50	Burn remaining payment and matching funds in channel 2 for 26.519-sat send	3	6.667 18.399 18.982 0.583 3.240	6.667 13.640 14.223 0.583 7.999	6.667 2.962 3.545 0.583 18.677			20.000 35.000 36.750 1.750
51	Send 26.519 sats	3	6.667 8.481 18.982 10.501 3.240	6.667 0.000 14.223 14.223 7.999	6.667 0.000 3.545 3.545 18.677			20.000 8.481 36.750 28.269
52	Move funds in channel 2 from burn outputs for 26.519-sat send	3	6.667 8.481 18.982 10.501 3.240	6.667 0.000 0.000 0.000 22.222	6.667 0.000 3.545 3.545 18.677			20.000 8.481 22.527 14.046
53	Move funds in channel 3 from burn outputs for 26.519-sat send	3	6.667 8.481 18.982 10.501 3.240	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222			20.000 8.481 18.982 10.501
54	Move funds in channel 1 from burn outputs for 26.519-sat send	3	15.147 0.000 0.000 0.000 13.741	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222			28.481 0.000 0.000 0.000
55	Burn payment and matching funds in channel 1 for 3.481-sat send	3	10.554 4.594 4.824 0.230 13.512	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222			23.887 4.594 4.824 0.230
56	Send 3.481 sats	3	10.554 1.113 4.824 3.710 13.512	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222			23.887 1.113 4.824 3.710
57	Move funds in channel 1 from burn outputs for 3.481-sat send	3	11.667 0.000 0.000 0.000 17.222	6.667 0.000 0.000 0.000 22.222	6.667 0.000 0.000 0.000 22.222			25.000 0.000 0.000 0.000

Table 6. Example of user U sending a Lightning payment of 30 sats. The 30-sat send can't be performed as a single atomic Lightning payment, so it's divided into two atomic Lightning payments.

5.3.3 Draining The Channels

Finally, U drains their channels prior to the depot's expiry. Assume U does this by sending U's remaining 25-sat balance to U's channels in another depot which atomically revokes U's 3 remaining active channels (as shown in Figure 6). Specifically, U first gives O adaptor signatures, with adaptor X , for index signatures $InSig_i$, $InSig_j$ and $InSig_k$. O then makes a 25-sat Lightning payment to U's channels in the other depot, where the secret for the payment, X , equals the adaptor for the index signatures. O can then use the receipt for the payment, X , to complete the adaptor signatures and revoke U's remaining active channels (by grinding through all P possible guess values for each channel, if necessary). However, assume U follows the protocol, so after U receives the 25-sat payment in the other depot, U gives O the values Gu_i , Gu_j and Gu_k .

The status of U's channels after revoking channels 1, 2 and 3 is shown in Step 58 in Table 7 below.

Step	Notes	Au	Channel 1	Channel 2	Channel 3	Channel 4	Channel 5	Total
58	Atomic revocation of channels 1, 2 and 3 with 25-sat send	0						0.000 0.000 0.000 0.000

Table 7. Example of user U draining and revoking their channels.

5.4 Aggregate Depot Analysis

Assume:

- 10,000,000,000 users who use depots,
- each user is allocated Lightning channels in a new depot with the above parameters (except the depot's value parameter, D) every 10,000 blocks (about 2.5 months),
- each user is allocated 10 channels in each depot,
- each Lightning payment to a depot requires a user to purchase an average of 1.821 new channels²⁷, and
- 1% of users fail to drain but remain active and attempt to put their channel on-chain (or burn the depot's outputs if there are multiple hits) and these users each have half (5) of their 10 allocated channels active at the expiry.

Given these assumptions, in the steady state there are:

²⁷ This is a conservative assumption, as it's based on simulations using the depot.py program [Law26] in which each send or receive changes the user's balance to a uniformly selected random value between the minimum (nonzero) and maximum balances the user can have in the depot. In practice, it's likely that most payments will result in smaller percentage changes in the user's balance, thus resulting in the user being able to purchase fewer new channels.

- 10,000,000,000 users each with Lightning channels in $(100,000 \text{ blocks}) / (10,000 \text{ blocks}) = 10$ different depots for a total of 100,000,000,000 Lightning channels,
- $10,000,000,000 \text{ users} / (10,000 \text{ blocks} * 1,000,000 \text{ users/depot}) = 1$ depot per block,
- 2.105 on-chain depot transactions per block on average²⁸ (using a small fraction of the available block space), and
- $1,000,000,000 \text{ cumulative channels/depot} * 1 \text{ depot/block} * 1/1.821 \text{ payment/channel} = 549,160,882 \text{ payments/block} = 915,268 \text{ payments/second}$.

5.5 Aggregate Routing Channel Analysis

In addition to the depots that create payment channels for 10,000,000,000 casual users, there's a need for routing channels that route the payments to and from depots. As was noted in Section 4.5, timeout trees are well-suited to the creation of routing channels.

Assume:

- 10,000 depot operators, each of whom also co-owns at least 200 routing channels,
- each timeout tree has 1,000 leaves, each of which supports either a 3-party hierarchical channel **[Law23a]** with one 2-party routing channel or a 4-party hierarchical channel **[Law23a]** with two 2-party routing channels,
- each timeout tree has a lifetime of 1,000 blocks (about a week),
- one timeout tree is created in each block,
- each leaf node that's put on-chain requires an average of 15 on-chain transactions,
- each payment traverses an average of 3 routing channels, and
- each payment takes an average of 3 seconds to complete.

In order to support the depots' full 892,220 payments/second, the routing channels must support an average of $(892,220 \text{ payments/second}) * (3 \text{ seconds/payment}) \approx 2,677,000$ active payments in the steady state. Each of these active payments traverses 3 routing channels on average for a total of approximately 8,030,000 active PTLCs in the steady state, spread across the $(1,000 \text{ timeout trees}) * (1,000 \text{ routing channels per timeout tree}) = 1,000,000$ routing channels for a total of $8,030,000 / 1,000,000 \approx 8$ active PTLCs per routing channel in the steady state.

Of course, if the standard Lightning channel protocol were used to route so many payments, the need to occasionally resolve a PTLC on-chain would likely exceed the block space available. However, as

²⁸ The program depot.py **[Law26]** can be used to calculate the expected number of on-chain transactions for an arbitrary depot.

noted in Section 4.5 the Off-chain Payment Resolution (OPR) protocol [Law24] can be used to route these payments without requiring any additional on-chain transactions, thus eliminating this concern.

In addition, the routing channels may have to be rebalanced frequently in order to assure adequate capital in both directions. The majority of this rebalancing should be able to be done entirely off-chain by using the hierarchical channel protocol [Law23a]. Furthermore, the short lifetime of each routing channel provides a means for creating a new, rebalanced routing channel every week. Finally, even if a routing channel isn't rebalanced, it can still be used for routing in at least one direction. As a result, the majority of routing channels should be able to remain entirely off-chain.

It's beyond the scope of this paper to fully quantify the rate at which routing channels will be put on-chain. However, as an example assume an average of 10 leaf nodes per timeout tree (1% of the total) are put on-chain. In this case, each timeout tree requires 1 transaction to fund the timeout tree and an average of $(10 \text{ on-chain leaf nodes/timeout tree}) * (15 \text{ on-chain transactions/on-chain leaf node}) = 150$ on-chain transactions for leaves that are put on-chain, for a total of 151 on-chain transactions per timeout tree. Given that one timeout tree is created per block, such timeout trees would contribute an average of 151 on-chain transactions per block, representing a small fraction of the total block space.

6 Improvements And Extensions

6.1 Dynamic Channel Allocation And Reallocation

The protocol given in Section 3.3 requires that the users of the depot and each user's channel allocation be known when the depot is created (in order to be included in the merkle-sum tree defined by the Funding transaction output's Taproot internal key) and they can't be changed during the depot's lifetime. However it's easy to modify the protocol to support dynamic channel allocations and reallocations.

6.1.1 Dynamic Channel Allocations

First, if some of the channel allocations aren't assigned to users when the depot is created, the depot's merkle-sum tree can have one or more leaves that specify that those channels will be allocated to users in the future. To do this, the leaf has the user identifier "UNALLOCATED" and a reference to the future on-chain transaction (e.g., identified by the outpoint that it spends) which will allocate the channels to a user. That future transaction can then have an output with a Taproot internal key that specifies the allocation of the channels to a user. However, that future transaction can likewise use the "UNALLOCATED" identifier to further delay the allocation of some or all of the channels to users. This technique can be repeated as many times as necessary, as long as all users will follow the same chain of transactions to determine the same allocation of channels to users.

6.1.2 Dynamic Channel Reallocations

In addition, if a user U no longer needs their channel allocation, U can deallocate their channels, thus allowing O to reallocate them to another user. In order to deallocate their channels, U signs a message consisting of the depot's ID (that is, the outpoint of the UTXO that funds it), the ID of the channels

allocated to U (that is, the path in the merkle-sum tree that allocates the channels to U), and the string "DEALLOCATE".

The deallocation can be incentivized by requiring U to pay O a fee for the cost of capital allocated to U for the depot's lifetime and then refunding the portion of that fee corresponding to the amount of time between the deallocation and the depot's expiry. This refund can be included in the payment to U when U drains their funds from the depot, and the refund can be made atomic with the deallocation by having U provide an adaptor signature for the deallocation with an adaptor that matches the payment's PTLT secret (and the adaptor for the revocation of U's last active channels).

Once U has deallocated their channels, O can reallocate them to another user V by 1) giving V U's deallocation signature, and 2) announcing the reallocation to V by including it in a merkle-sum tree that's committed to in a pre-specified sequence of locations (e.g, specified by using a chain of transactions, each of which has an output with a Taproot internal key that specifies a merkle-sum tree and a link to the next transaction in the chain). The only requirement is that the reallocation to V must be such that it's found by any other user W so that the same allocation can't be given to more than one user at a time. Note that the reallocation can be done at a dynamically selected time provided that any other user W checks all of the possible locations for the reallocation notice.

Finally, note that if V doesn't need the channels reallocated to them for the entire remainder of the depot's lifetime, V can dynamically deallocate them and allow them to be reallocated again by O. As a result, the depot's capital can be allocated and deallocated dynamically throughout the depot's lifetime via commitments and off-chain signatures that require no additional on-chain bytes. The only cost for these dynamic allocations and deallocations is the requirement for a new user who acquires reallocated channels to verify signatures showing that all previous allocations have been deallocated and to scan all possible locations for merkle-sum trees that could contain reallocations of those channels²⁹.

6.2 Incentivizing Draining

The depot protocol requires users to drain their funds, with the operator's cooperation, prior to the expiry. The depot protocol provides an incentive to drain funds, as the expected value of one's funds left in a depot after its expiry can be less than (but not more than) the cost of those funds. This incentive can be strengthened by making users and the operator put a small fraction of their funds in a burn output in an active Lightning channel.

7 Limitations

Depots have a number of limitations:

- **Reliance on griefer penalization:** The security of a user's or operator's funds in a depot is not guaranteed, as one's funds could be lost due to a griefing attack. However, griefing attacks are penalized, so if one only partners with self-interested parties one will not be attacked.

²⁹ Of course, O must provide the paths in those merkle-sum trees to the new user so the new user can examine them.

Furthermore, the ability of the current Lightning protocol to route payments that are smaller than the on-chain fees required to claim them is strong evidence that even unpenalized griefing attacks are rare [Harding24], so penalized griefing attacks should be even rarer.

- **Reliance on expected values:** Rather than guaranteeing that one will receive the full value of one's funds in a depot, the depot protocol guarantees that the expected value one receives equals one's funds in a depot (unless there is a griefing attack or a party fails for an extended period of time). This is a weaker guarantee and many people would prefer to avoid the risk, especially for large balances (hence the existence of insurance against large losses). However, the replacement of a fixed balance by an expected value only occurs if one's partner performs a griefing attack (which should be very rare as explained above) or has a lengthy failure (e.g., for 2.5 months).
- **Potential to delay access to users' funds:** Because users can only send and receive Lightning payments to and from a depot prior to its expiry, a malicious operator could choose to block users' access to their funds until shortly before the depot's expiry. Such a delay in no way benefits the operator and it doesn't cause users to lose funds. The lack of benefit to the operator and the resulting loss of the operator's reputation should minimize the frequency of such attacks.
- **Requirement to fund from and drain to the user's Lightning channel:** In order to deposit a user's initial funds in a depot, the user must transfer those funds from another Lightning channel they control (rather than receive a Lightning payment from another user) or they must rely on a trusted third party who will provide them with the payment's receipt. This requirement complicates the initial on-boarding of a user to a depot, as that user must first acquire a Lightning channel (e.g., in a timeout tree) or rely on a trusted third party, such as an exchange. Similarly, in order to drain their funds from a depot, a user must transfer those funds to another Lightning channel they control (rather than make a Lightning payment to another user) or trust the depot's operator to make a Lightning payment on the user's behalf.
- **Requirement for active draining:** The depot protocol requires active participation from users as they must drain their funds prior to the depot's expiry. However, this is a minimal requirement for interactivity, as a user can drain at any time with the only cost being the cost of capital for depot funds allocated to the user between the time of the drain and the depot's expiry. For example, if a depot's expiry is 100,000 blocks after its funding, a user can choose to drain within the last 10,000 blocks prior to the depot's expiry. This gives the user about 2.5 months in which to drain and results in at most a 10% (10,000 blocks / 100,000 blocks) cost of capital charge. Furthermore, the cost of unused capital can be reduced by dynamically reallocating the first user's channels (see Section 6.1) to another user with greater availability and interactivity (e.g., one who's willing to wait until the final 1,000 blocks before the depot's expiry before draining). Finally, it's worth noting that if a user drains to a depot operated by the same operator as the current depot, the drain's Lightning payment doesn't need to traverse any routing nodes

and thus can't be blocked by a third party. Because such a drain only depends on the user's and operator's availability, it can be performed shortly before the depot's expiry.

- **Lower bound on a user's nonzero balance in a depot:** Whenever a user has a nonzero balance in a depot, the user must have a certain number of active channels³⁰, each of which must have a minimum user balance. As a result, there's a lower bound on the amount a user can deposit in a depot in order to use the depot. For example, in the depot presented in Section 5.1, a typical user with an allocation of 10 channels must fund one channel with at least 6.667 sats. However, depots can be created that support extremely small per-user balances, thus yielding extremely small (e.g. less than 1 sat) on-boarding and minimum balance requirements.
- **Bounds on the size of large atomic sends and receives:** In order to atomically send funds to (or receive funds from) a third party, a user must own a number of active channels that's compatible with the user's balance throughout the processing of the payment. Thus, if there is no number of active channels that's compatible with payment's lowest and highest user balances, the user has to break that payment into multiple smaller payments, each of which has a separate receipt. An example is shown in the 30-sat send described in Table 6. Such multi-part payments should be relatively rare, as it's likely that most payments will result in a small percentage change in the user's balance³¹. In addition, if a user has balances in multiple depots, an atomic multi-path payment could be made to or from those depots in order to obtain a payment with a single receipt.
- **Bounds on the number of sends and receives per depot:** Unlike a Lightning channel that's on-chain, in a channel factory or in a timeout tree, a depot can't be used to make an unbounded number of sends and receives, because some sends and receives require creating or revoking channels, which are a limited resource. This limitation is a key reason that depots are not well-suited to implementing routing channels. On the other hand (as was shown in Section 5.4), depots can provide billions of users the ability to continuously send and receive hundreds of thousands of payments per second while occupying only a small fraction of the block space.
- **Capital inefficiency due to putting matching funds in burn outputs:** Payments to and from depots require that both the user and the operator put matching funds into a burn output (in addition to the funds for the payment itself). These matching funds are capital that can't be used for other purposes, thus creating some inefficiency in the use of capital. Fortunately, this inefficiency is relatively small (e.g., 38.6% of the payment value for the example considered in Section 5.3) and it only lasts for the duration of the payment.

30 The required number of active channels is given by the value *min_nonzero_channels* calculated by the *depot.py* program [Law26].

31 These multi-part payments can be made rarer by choosing a larger value for the *balance_range* parameter, but there's a trade-off between the *balance_range* and the depot's capital efficiency (as quantified by its *maximum_utilization* value).

- **Capital inefficiency due to the inability to allocate all depot funds to users:** In order to penalize griefing attacks in which the operator grieves the users by preventing them from draining their funds from the depot, the operator can't allocate all of the depot's funds to users, thus causing some capital inefficiency. For example, given the parameters in Section 5.1, the operator is able to allocate 92.83% of the depot's funds to users. While this represents a modest inefficiency, larger values of the griefing penalty parameter *GrO* and the *balance_range* parameter can lead to larger inefficiencies.
- **Capital inefficiency due to determining the existence of hits:** After a depot's expiry, the operator puts a Target transaction on-chain and waits to see if there are any hits. The maximum delay required for determining if there are hits depends on the depot's *tsd* (*to_self_delay*) safety parameter. In the case of no hits (which should be the most common case by far), the operator is able to reclaim all funds after a single *tsd* delay. If there is a hit, a delay of $3*tsd$ is required before the operator can spend the Hit transaction's output 0 and a delay of *tsd* before a user can put a Claim transaction on-chain.
- **Loss of funds due to burning:** Depots use burn outputs to secure payments and Burn transactions to penalize the failure to drain balances prior to the depot's expiry. If all parties follow the protocol, no funds will ever be burned. However, there's the possibility that a party will engage in a griefing attack (despite being penalized for doing so) or will have a failure that causes funds to be burned. Two possible reasons for burning funds stand out as being the most likely. First, timing errors and/or crashes could cause the parties to disagree on the resolution of a payment, thus causing the payment amount and its matching funds to be burned. Techniques to minimize such burns are presented in Section 4.2.3. Second, users could intentionally fail to drain their funds from a depot in order to have a very small chance of winning a large reward (despite being penalized for doing so). While such failures to drain are possible, there are many other ways for users to get a small chance of winning a large amount that are more attractive because they are more immediate (e.g., lotteries, horse races, stock markets and prediction markets) and/or have better payoffs (e.g., stock markets and futures options).
- **Requirement for multiple signatures for a single payment:** Because some payments require updating multiple channels, and some channels are updated multiple times for a single payment, the compute requirements for generating signatures are larger in depots than in on-chain Lightning channels. For example, the 30-sat send described in Table 6 requires 13 channel updates (plus channel revocations and the logical reallocation of burn funds from one party to another, which don't require channel updates and signatures). Overall, the example given in Sections 5.1 and 5.2 requires an average of just under 12 channel updates per send or receive³².

32 This is a conservative assumption, as it's based on simulations using the depot.py program [Law26] in which each send or receive changes the user's balance to a uniformly selected random value between the minimum (nonzero) and maximum balances the user can have in the depot.

8 Related Work

Depots are based on a number of previously-published ideas.

The idea of a time-limited output that's funded and put on-chain by an operator for a large number of users was presented by Law **[Law21][Law23b]** (in the context of timeout trees), Keceli **[Keceli23]** (in the context of Ark) and ZmnSCPxj **[ZmnSC24]** (in the context of SuperScalar).

Rivest proposed replacing a small deterministic output with a probabilistic output with the same expected value in order to reduce processing overhead in the context of micropayments **[Rivest97]** and Caldwell made a similar proposal for small bitcoin payments **[Cal12]**.

The idea of using a burn output to encourage channel partners to cooperate was introduced by Riard **[Riard22]** (in the context of unjamming Lightning) and utilized by Law **[Law24][Law25]** (in the context of resolving payments off-chain and unjamming Lightning).

Finally, the depot protocol has some commonalities with the Last Mile protocol **[Hynek24]**, as both protocols achieve security by appealing to each party's self-interest. However, the Last Mile protocol differs from the depot protocol in many ways. For example, the Last Mile protocol motivates cooperation by having both parties contribute to fees that are paid immediately to a miner (rather than putting funds in a burn output) and it requires constant monitoring of the mempool so that high-fee transactions can be sent to miners in time to get them included in the blockchain (as opposed to giving users multiple months in which to respond to on-chain transactions).

9 Conclusions

Bitcoin allows anyone to hold their own funds and to make safe, censorship-resistant online payments. However, due to block space limitations, only a tiny fraction of the world's payments can be performed on-chain. Protocols for Lightning channels, channel factories, timeout trees, Ark and SuperScalar have been proposed for improving Bitcoin's scalability by allowing most payments to be made off-chain. These protocols greatly decrease the number of on-chain transactions (at least in the common case), but they don't allow everyone in the world to make frequent casual online payments.

First, all of the above protocols give users transactions that they may need to put on-chain in order to prevent theft. Therefore, if a user doesn't own enough bitcoin to pay the fees for putting a transaction on-chain, the above protocols don't allow the user to prevent the theft of their funds.

Second, all of the above protocols prevent theft by forcing users to put certain transactions on-chain within a bounded number of blocks (or time). While the protocols are designed to keep these theft-preventing transactions off-chain in the common case, there's no guarantee that they won't have to be put on-chain. In fact, once the number of theft-preventing transactions held off-chain exceeds the block space available for putting them on-chain within the required window, the protocols' safety guarantees disappear **[Towns23][Carval26]**. As a result, rather than providing safe long-term scaling solutions,

these protocols provide safe scaling of payments until their success undermines their safety, at which point a catastrophic breakdown and massive theft become possible³³.

This paper presents depots that allow large numbers of casual users to self-custody bitcoin in a theft-free manner and to make a huge number of casual, censorship-resistant online payments. Depots don't suffer from either of the limitations described above, as they prevent theft even for users who own tiny amounts of bitcoin and they don't create more off-chain transactions than can be put on-chain before theft occurs. Depots also support on-boarding new users with balances that are a small fraction of a sat. As a result, depots have the potential to transform Bitcoin into a widely-used means of payment.

Depots require the CTV (CheckTemplateVerify) **[BIP-119]** and CSFS (CheckSigFromStack) **[BIP-348]** **[Roose25]** enhancements to the Bitcoin protocol. In addition, depots are much more efficient if the PC (PairCommit) **[BIP-PC]** **[Black24]**, multiply (OP_MUL) **[Russell24]** and mod (OP_MOD) operators **[Russell24]** are supported. It's hoped that the ability to support depots will help to motivate these enhancements to the Bitcoin protocol.

10 Acknowledgments

The author would like to thank David Harding for bringing to the author's attention previous work on probabilistic low-value payments **[Harding24]** and Clara Shikhelman for noting the connection between the Ultimatum Game and the behavior of Lightning users.

References

- BIP-119** BIP-119, "BIP 119: OP_CHECKTEMPLATEVERIFY", <https://github.com/bitcoin/bips/blob/master/bip-0119.mediawiki>
- BIP-348** BIP-348, "BIP 348: OP_CHECKSIGFROMSTACK", <https://github.com/bitcoin/bips/blob/050d422b2ac24d8221edab0ff0053e0f585409f7/bip-0348.md>
- BIP-PC** BIP-PC, "OP_PAIRCOMMIT", <https://github.com/bitcoin/bips/pull/1699>
- Black24** Black, "LNHANCE bips and implementation", <https://delvingbitcoin.org/t/lnhance-bips-and-implementation/376>
- BR24** Black and Rubin, "CSFS Re-Keying and Laddering, Deterministic Update Rekeying, & Applications to LN-Symmetry", <https://rubin.io/bitcoin/2024/12/02/csfs-ctv-rekey-symmetry/>
- Cal12** Caldwell, "Sustainable nanopayment idea: Probabilistic Payments", <https://bitcointalk.org/index.php?topic=62558.0>
- Carval26** Carvalho, "Conservation of Blockspace", <https://blockspace.science/>
- CPFP** CPFP, "Child pays for parent (CPFP)", <https://bitcoinops.org/en/topics/cpfp/>

³³ An alternative approach to securing these protocols as they scale is to introduce Feerate-Dependent Timelocks (FDTs) **[Law23c]**. FDTs don't provide bounds on worst-case on-chain transactions (which depots do provide), but they reduce or eliminate the risk of theft in case of worst-case behavior, so they largely remove the incentive for such a worst case.

GKP94 Graham, Knuth and Patashnik. Concrete Mathematics, Second Edition. 1994. Addison-Wesley: Upper Saddle River, NJ.

Gowers08 Gowers. The Princeton Companion To Mathematics. 2008 Princeton University Press: Princeton, NJ.

Harding24 Harding, Response to: "A Fast, Scalable Protocol For Resolving Lightning Payments", <https://delvingbitcoin.org/t/a-fast-scalable-protocol-for-resolving-lightning-payments/1233/3>

Hynek24 Hynek, "Last Mile Transactions", <https://github.com/hynek-jina/Hynek/blob/0301c90015f9d66fb22e347f21924dc9d04f0bc8/blog/Last%20mile%20transactions.md>

Keceli23 Keceli, "Ark: An Alternative Privacy-preserving Second Layer Solution", <https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2023-May/021694.html>

Law21 Law, "Scaling Bitcoin With Inherited IDs", <https://github.com/JohnLaw2/btc-iids>

Law23a Law, "Resizing Lightning Channels Off-Chain With Hierarchical Channels", <https://github.com/JohnLaw2/ln-hierarchical-channels>

Law23b Law, "Scaling Lightning With Simple Covenants", <https://github.com/JohnLaw2/ln-scaling-covenants>

Law23c Law, "Scaling Lightning Safely With Feerate-Dependent Timelocks", <https://github.com/JohnLaw2/ln-fdts>

Law24 Law, "A Fast, Scalable Protocol For Resolving Lightning Payments", <https://github.com/JohnLaw2/ln-opr>

Law25 Law, "Fee-Based Spam Prevention For Lightning", <https://github.com/JohnLaw2/ln-spam-prevention>

Law26 Law, "Depots: Theft-Proof, Self-Custodial Bitcoin For Billions Of Users" and the associated programs depot.py, burn_depot.py and coupon.py, <https://github.com/JohnLaw2/ln-depots>

Moreh24 Morehouse, Response to: "A Fast, Scalable Protocol For Resolving Lightning Payments", <https://delvingbitcoin.org/t/a-fast-scalable-protocol-for-resolving-lightning-payments/1233/2>

Nak09 Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System", <http://bitcoin.org/bitcoin.pdf>

Osunto21 Osuntokun, "Taro: A Taproot Asset Representation Overlay", <https://github.com/Roasbeef/bips/blob/bip-taro/bip-taro.mediawiki>

Riard22 Riard, "Unjamming lightning (new research paper)", <https://www.mail-archive.com/lightning-dev@lists.linuxfoundation.org/msg02996.html>

Rivest97 Rivest, "Electronic Lottery Tickets As Micropayments", <https://people.csail.mit.edu/rivest/pubs/Riv97b.pdf>

Roose25 Roose, "CTV+CSFS: Can we reach consensus on a first step towards covenants?", <https://delvingbitcoin.org/t/ctv-csfs-can-we-reach-consensus-on-a-first-step-towards-covenants/1509>

Russell24 Russell, "Restoring Bitcoin's Full Script Power",
<https://rusty.ozlabs.org/2024/01/19/the-great-opcode-restoration.html>

Towns23 Towns, "Re: Scaling Lightning With Simple Covenants",
<https://lists.linuxfoundation.org/pipermail/bitcoin-dev/2023-September/021943.html>

Ultimatum "Ultimatum Game", https://en.wikipedia.org/wiki/Ultimatum_game

ZmnSC24 ZmnSCPj, "SuperScalar: Laddered Timeout-Tree-Structured Decker-Wattenhofer Factories With Pseudo-Spilman Leaves", <https://delvingbitcoin.org/t/superscalar-laddered-timeout-tree-structured-decker-wattenhofer-factories-with-pseudo-spilman-leaves/1242>

Appendix A. Quantified Griefer Penalization

Let:

- $LossO$ = the operator's expected loss,
- $LossU$ = the sum of the expected losses of all users, and
- GrO and GrU denote the griever penalization security parameters such that at all times:
 - $\frac{LossO}{LossU} \geq GrO$ and
 - $\frac{LossU}{LossO} \geq GrU$
 - where $0 < GrO$, $0 < GrU$ and $GrO * GrU < 1$.

While the parameters GrO and GrU provide an intuitive way of expressing the griever penalization requirements, there's an equivalent formulation that leads to simpler security proofs.

Let:

- $LossT = LossO + LossU$ = the total expected loss of the operator and of all users,
- $FracLossU$ = the ratio $\frac{LossU}{LossT}$,
- $MinFracLossU = \frac{GrU}{1+GrU}$, and
- $MaxFracLossU = \frac{1}{1+GrO}$.

Theorem 1: The requirement $\frac{LossO}{LossU} \geq GrO$ is equivalent to $FracLossU \leq MaxFracLossU$ and the requirement $\frac{LossU}{LossO} \geq GrU$ is equivalent to $FracLossU \geq MinFracLossU$.

Proof:

Note that $\frac{LossO}{LossU} \geq GrO$ is equivalent to $LossO \geq GrO * LossU$ which is equivalent to

$LossO + LossU \geq (1 + GrO) * LossU$ which is equivalent to

$FracLossU = \frac{LossU}{LossO + LossU} \leq \frac{LossU}{(1 + GrO) * LossU} = \frac{1}{1 + GrO} = MaxFracLossU$. Similarly, note that

$\frac{LossU}{LossO} \geq GrU$ is equivalent to $LossU \geq GrU * LossO$ which is equivalent to

$LossO + LossU \geq (1 + GrU) * LossO$ which is equivalent to

$FracLossU = 1 - \frac{LossO}{LossO + LossU} \geq 1 - \frac{LossO}{(1 + GrU) * LossO} = 1 - \frac{1}{1 + GrU} = \frac{GrU}{1 + GrU} = MinFracLossU$.

□

As a result of Theorem 1, in the remaining appendices sufficient griever penalization will be proven by showing that $MinFracLossU \leq FracLossU \leq MaxFracLossU$.

Appendix B. Notation And Formulas

This appendix presents notation and formulas required for operating depots and for proving that depots satisfy their quantitative griever penalization security requirements. This notation will be used throughout the remaining appendices.

Let:

- D = the depot's total value (that is, the value of the Funding transaction's output),
- n denote the total number of users in the depot,
- P denote the large prime such that each active channel has a $1/P$ chance of being a hit,
- Au = number of currently active channels for user u ,
- $A = \sum_{u=1}^n Au$ = the total number of currently active channels across all users,
- $MaxAu$ = maximum number of active channels allocated to user u ,
- $MaxA = \sum_{u=1}^n MaxAu$ = the total maximum number of active channels allocated to all users,
- $MaxC$ = maximum cumulative number of channels created over the lifetime of the depot,
- $H = \frac{A}{P} = \sum_{u=1}^n \left(\frac{Au}{P} \right)$ = the current expected number of hits,

- $MaxH = \frac{MaxA}{P}$ = the maximum expected number of hits,
- $Hu = (\frac{Au}{MaxAu}) * (MaxH) = (\frac{Au}{MaxAu}) * (\frac{MaxA}{P}) = \frac{Au * MaxA}{MaxAu * P}$ = the expected number of hits if all users have user u 's fraction of their allocated channels currently active,
- Bu = user u 's current balance,
- $B = \sum_{u=1}^n Bu$ = the current total balance of all users,
- $UserChannelMin = \frac{D * MinFracLossU}{2 * P}$ = the lower bound on a user's balance in any active channel,
- $MinB(Au) = Au * UserChannelMin$ = the lower bound on a user's total depot balance as a function of u 's current number of active channels,
- $MinBNoBurn(Au) =$
 - 0 if $Au = 0$ and
 - $MinB(Au) + \frac{MinFracLossU}{MaxFracLossU - MinFracLossU} * UserChannelMin$ if $1 \leq Au \leq MaxAu$
 - is the lower bound on a user's total depot balance when the user has no contribution to a burn output as a function of u 's current number of active channels³⁴,
- $f(x) = \frac{e^x - 1 - x}{x(e^x - 1)},$
- $g(x) = \frac{e^x - 1 - x}{e^x - 1},$
- $MaxB(Au, MaxAu) = \frac{D * MaxFracLossU * MaxAu * g(Hu)}{MaxA}$ = the upper bound on user u 's balance as a function of u 's current number of active channels and the number of active channels allocated to u ,
- $Capacity(i, MaxAu) =$
 - $MaxB(1, MaxAu)$ if $i = 1$ and
 - $max(UserChannelMin, MaxB(i, MaxAu) - MaxB(i - 1, MaxAu))$ if $2 \leq i \leq MaxAu$

34 The additive term represents u 's matching contribution for a payment of size $UserChannelMin$, as will be shown in Appendix F.

- is the per-channel capacity (maximum balance) of user u 's i -th oldest channel, and
- $OperatorChannelMin = \frac{D * (1 - MaxFracLossU)}{2 * P}$ = the lower bound on the operator's balance in any active channel.

Appendix C. Security Against Operator's Failure-To-Drain Attack

C.1 Technical Results

This subsection provides technical results related to a failure-to-drain griefing attack by the operator, in which O prevents one or more users from draining their funds from the depot prior to its expiry.

A function $h(x)$ is **concave** if its second derivative, $h''(x)$, is negative.

Jensen's Inequality ([Gowers08], p. 704): If a function $h(x)$ is concave, then given any nonnegative λ_i 's such that $\lambda_1 + \dots + \lambda_n = 1$ and any $x_1 \dots x_n$, $h(\lambda_1 * x_1) + \dots + h(\lambda_n * x_n) \leq h(\lambda_1 * x_1 + \dots + \lambda_n * x_n)$.

Recall that $g(x) = \frac{e^x - 1 - x}{e^x - 1}$.

Lemma 1: For all $x > 0$, $g''(x) < 0$.

Proof: Let $gnum(x) = e^x - 1 - x$ and $gden(x) = e^x - 1$ and note that $gnum'(x) = e^x - 1$ and $gden'(x) = e^x$. Therefore

$$g'(x) = \frac{(e^x - 1) * (e^x - 1) - (e^x - 1 - x) * e^x}{(e^x - 1)^2} = \frac{e^{2x} - 2e^x + 1 - e^{2x} + e^x + xe^x}{(e^x - 1)^2} = \frac{xe^x - e^x + 1}{(e^x - 1)^2}.$$

Let $gpnum(x) = xe^x - e^x + 1$ and $gpden(x) = (e^x - 1)^2 = e^{2x} - 2e^x + 1$ and note that $gpnum'(x) = e^x + xe^x - e^x = xe^x$ and $gpden'(x) = 2e^{2x} - 2e^x$.

Therefore $g''(x) = \frac{(xe^x) * (e^{2x} - 2e^x + 1) - (xe^x - e^x + 1) * (2e^{2x} - 2e^x)}{(e^x - 1)^4}$ which equals

$$\frac{xe^{3x} - 2xe^{2x} + xe^x - 2xe^{3x} + 2xe^{2x} + 2e^{3x} - 2e^{2x} - 2e^{2x} + 2e^x}{(e^x - 1)^4} \text{ which equals}$$

$$\frac{(2-x)e^{3x} - 4e^{2x} + (2+x)e^x}{(e^x - 1)^4} = \frac{e^x}{(e^x - 1)^4} * a(x) \text{ where } a(x) = (2-x)e^{2x} - 4e^x + 2 + x.$$

Assume for the sake of contradiction that there exists an $x > 0$ such that $g''(x) \geq 0$.

Note that for all $x > 0$, $\frac{e^x}{(e^x-1)^4} > 0$, so $g''(x) \geq 0$ implies that $a(x) \geq 0$. Note that

$a(1) = e^2 - 4e + 3 \approx -0.4841 < 0$ and $a(x)$ is continuous, so $g''(x) \geq 0$ implies that there exists an $x > 0$

such that $a(x) = 0$. Using the Taylor series expansion $e^{kx} = \sum_{i=0}^{\infty} \frac{(kx)^i}{i!}$,

$$a(x) = 2 + x + \sum_{i=0}^{\infty} \frac{2^{i+1} * x^i - 2^i * x^{i+1} - 4 * x^i}{i!} = \sum_{i=1}^{\infty} \frac{2^{i+1} * x^i - 2^i * x^{i+1} - 4 * x^i}{i!}.$$

Therefore $a(x) = 0$ implies $\sum_{i=1}^{\infty} \frac{4 * x^i + 2^i * x^{i+1}}{i!} = \sum_{i=1}^{\infty} \frac{2^{i+1} * x^i}{i!}$. Therefore

$$\sum_{i=1}^{\infty} \frac{2^i * x^{i+1}}{i!} = \sum_{i=1}^{\infty} \frac{(2^{i+1} - 4) * x^i}{i!} \quad \text{and} \quad \sum_{i=1}^{\infty} \frac{2^i * x^{i+1}}{i!} = \sum_{i=2}^{\infty} \frac{(2^{i+1} - 4) * x^i}{i!} \quad \text{so} \quad \sum_{i=2}^{\infty} \frac{2^{i-1} * x^i}{(i-1)!} = \sum_{i=2}^{\infty} \frac{(2^{i+1} - 4) * x^i}{i!}$$

and $\sum_{i=2}^{\infty} \frac{i * 2^{i-1} * x^i}{i!} = \sum_{i=2}^{\infty} \frac{(2^{i+1} - 4) * x^i}{i!}$ and $\sum_{i=2}^{\infty} \frac{(2^{i+1} - i * 2^{i-1} - 4) * x^i}{i!} = 0$ and

$$\sum_{i=4}^{\infty} \frac{(2^{i+1} - i * 2^{i-1} - 4) * x^i}{i!} = 0. \quad \text{However, note that for all } i \geq 4, \quad 2^{i+1} - i * 2^{i-1} - 4 < 0, \quad \text{so for any } x > 0,$$

$$\sum_{i=4}^{\infty} \frac{(2^{i+1} - i * 2^{i-1} - 4) * x^i}{i!} < 0, \quad \text{which is a contradiction. } \square$$

Recall that $MaxA = \sum_{u=1}^n MaxAu$, $Hu = \frac{Au * MaxA}{MaxAu * P}$ and $H = \sum_{u=1}^n \left(\frac{Au}{P}\right)$.

Theorem 2: $\sum_{u=1}^n \left(\left(\frac{MaxAu}{MaxA}\right) * g(Hu)\right) \leq g(H)$.

Proof: Note that $\sum_{u=1}^n \frac{MaxAu}{MaxA} = 1$. Lemma 1 shows that $g(x)$ is concave for all $x > 0$. Therefore, it follows from Jensen's inequality that

$$\sum_{u=1}^n \left(\left(\frac{MaxAu}{MaxA}\right) * g(Hu)\right) \leq g\left(\sum_{u=1}^n \left(\left(\frac{MaxAu}{MaxA}\right) * Hu\right)\right) = g\left(\sum_{u=1}^n \left(\frac{MaxAu * Au * MaxA}{MaxA * MaxAu * P}\right)\right) = g\left(\sum_{u=1}^n \left(\frac{Au}{P}\right)\right) = g(H).$$

$\square$

C.2 Analysis Of Operator's Failure-To-Drain Attack

In order to secure the depot against the operator's failure to allow users to drain, it's required that for each user u ,

$$Bu \leq MaxB(Au, MaxAu) = \frac{D * MaxFracLossU * MaxAu * g(Hu)}{MaxA}.$$

Therefore,

$$B = \sum_{u=1}^n Bu \leq \sum_{u=1}^n \left(\frac{D * \text{MaxFracLossU} * \text{MaxAu} * g(Hu)}{\text{MaxA}} \right) = D * \text{MaxFracLossU} * \sum_{u=1}^n \left(\frac{\text{MaxAu} * g(Hu)}{\text{MaxA}} \right).$$

Theorem 2 in Appendix C.1 proves that $\sum_{u=1}^n \left(\left(\frac{\text{MaxAu}}{\text{MaxA}} \right) * g(Hu) \right) \leq g(H)$.

Therefore, $B \leq D * \text{MaxFracLossU} * g(H)$.

Let

- pzh = the probability of exactly zero hits,
- poh = the probability of exactly one hit, and
- pmh = the probability of multiple (2 or more) hits.

Note that LossU is the total balance of all users minus the expected value of the users' active channels. Furthermore, the probability that an active channel will be put on-chain is poh and the average payout

to a user if an active channel is put on-chain is $\frac{B * P}{A}$, so

$$\text{LossU} = B - poh * \left(\frac{B * P}{A} \right) = \left(1 - \frac{poh * P}{A} \right) * B = \left(1 - \frac{poh}{H} \right) * B = \left(\frac{H - poh}{H} \right) * B.$$

Furthermore, depot funds are lost if and only if there is more than one hit, so $\text{LossT} = pmh * D$.

Combining these results yields $\text{FracLossU} = \frac{\text{LossU}}{\text{LossT}} = \left(\frac{H - poh}{H * pmh} \right) * \left(\frac{B}{D} \right)$.

Note that $\lim_{x \rightarrow \infty} \left(1 - \frac{1}{x} \right)^x = e^{-1}$ so $\left(1 - \frac{1}{P} \right)^P \approx e^{-1}$.

Therefore $pzh = (1 - 1/P)^A = (1 - 1/P)^{P * H} \approx e^{-H}$

and $poh = A * (1/P) * (1 - 1/P)^{A-1} = H * (1 - 1/P)^{(P * H) - 1} = \frac{H * (1 - 1/P)^{P * H}}{1 - 1/P} \approx H * e^{-H}$

so $pmh = 1 - pzh - poh \approx 1 - e^{-H} - H * e^{-H} = (e^H - 1 - H) * e^{-H}$.

Combining the above results yields

$$\text{FracLossU} = \left(\frac{H - poh}{H * pmh} \right) * \left(\frac{B}{D} \right) \approx \left(\frac{H - H * e^{-H}}{H * (e^H - 1 - H) * e^{-H}} \right) * \left(\frac{B}{D} \right) =$$

$$\left(\frac{e^H - 1}{e^H - 1 - H}\right) * \left(\frac{B}{D}\right) = \frac{B}{D * g(H)} \quad \text{so (to the level of accuracy of the approximate equalities above)}^{35}$$

$$\text{FracLossU} \approx \frac{B}{D * g(H)} \leq \frac{D * \text{MaxFracLossU} * g(H)}{D * g(H)} = \text{MaxFracLossU} \quad \text{and (from Theorem 1) users}$$

have the required security against the operator attempting to grief them by failing to allow them to drain prior to the depot's expiry.

Appendix D. Security Against Users' Failure To Drain

The depot protocol calls for users to burn the entire depot whenever there are two or more hits. However, this appendix examines an attack by users and attacking users may violate the protocol if doing so increases the success of their attack. As a result, this appendix considers the case where some or all of the attacking users violate the protocol by putting a channel on-chain even though there are multiple hits³⁶.

In order to simplify the analysis, assume for now that there is only a single target with target value t , and that all active channels are attempting to hit t . In the following analysis, the channels that are active and the effective guess of each active channel will be viewed as arbitrary and fixed, while the target value t will be viewed as a random variable.

Let

- nzc = the number of integers y , $0 \leq y < P$, for which there are exactly 0 active channels with effective guess y ,
- noc = the number of integers y , $0 \leq y < P$, for which there is exactly 1 active channel with effective guess y ,
- $nmctb$ ³⁷ = the number of integers y , $0 \leq y < P$, for which there are multiple (2 or more) active channels with effective guess y where the entire depot will be burned if $t = y$, and

³⁵ The approximations are extremely close for values of P that are at least 100,000. For example, if $P = 100,000$ then $(1 - (1/P))^P = 0.36787760$ and $e^{-1} = 0.36787944$ for a relative error of 0.0005%, and if $P = 1,864,361$ then $(1 - (1/P))^P = 0.36787934$ and $e^{-1} = 0.36787944$ for a relative error of 0.000027%. Furthermore, the approximations appear to (very slightly) increase the calculated value of FracLossU , in which case the desired inequality holds regardless of the error in the approximations.

³⁶ There are good reasons to expect the users to burn the entire depot whenever there are multiple hits, as failing to do so opens the users up to a failure-to-drain attack by the operator, to an operator who puts a Claim transaction for a fake hit on-chain, and to an operator who puts an Arbitrary transaction on-chain that spends the Hit transaction's output 1. Despite these reasons, this appendix includes the possibility of such attacks by users and when they occur it assumes (conservatively) that the attack always succeeds by allowing one of the attackers' Claim and Commitment transactions to be put on-chain.

³⁷ $nmctb$ stands for number-multiple-channel-total-burn values of y .

- $nmcpb^{38}$ = the number of integers y , $0 \leq y < P$, for which there are multiple (2 or more) active channels with effective guess y where the Hit transaction's output 0 will be burned and one of the channels with a hit will be put on-chain (in violation of the protocol) if $t = y$.

Note that $LossU$ is the sum, over all A active channels, of the users' balance in that channel minus the expected payout from that channel to a user. Each of the noc channels that have a unique effective guess have a $1/P$ chance of paying P times the channel's balance to a user, yielding zero expected loss to a user. Users have an expected loss of at least $UserChannelMin$ for each of the at least $2*nmctb$ channels that have an effective guess that matches the effective guess of another channel where the entire depot will be burned if the channel is a hit³⁹. Finally, users have an expected loss of at least $UserChannelMin$ for each of the at least $nmcpb$ channels that have an effective guess that matches the effective guess of another channel that will be put on-chain if the effective guess is a hit⁴⁰. Therefore, $LossU \geq (2*nmctb + nmcpb)*UserChannelMin = (2*nmctb + nmcpb)*MinFracLossU*D/(2*P)$.

Similarly, $LossT$ is the sum, over all target values y , $0 \leq y < P$, of the expected amount burned by the depot protocol for that target value⁴¹. Therefore, $LossT = nmctb*D/P + nmcpb*D/(2*P) = (2*nmctb + nmcpb)*D/(2*P)$.

Combining these results yields

$$FracLossU = \frac{LossU}{LossT} \geq \frac{(2*nmctb + nmcpb)*MinFracLossU*D*2*P}{2*P*(2*nmctb + nmcpb)*D} = MinFracLossU, \text{ and (from}$$

Theorem 1) the operator has the required security against the users' grieving attack consisting of failing to drain prior to the depot's expiry.

While the above analysis assumed that there is only a single target, it's straightforward to extend to the case of h separate targets. Specifically, repeat the above analysis for the channels that target each target value t_s where $1 \leq s \leq h$, and let $FracLossU_s$ denote the users' fraction of the expected loss for the channels that target t_s . The above analysis shows that $FracLossU_s \geq MinFracLossU$ for each of these h targets, so it follows that the overall $FracLossU \geq MinFracLossU$ (because any weighted average of numbers, each of which is at least $MinFracLossU$, is at least $MinFracLossU$).

Appendix E. Security Against Other Depot Attacks

Appendix C proved that if the operator performs a failure-to-drain attack and if all parties follow the protocol after the Targ or Targ' transaction is put on-chain, $FracLossU \leq MaxFracLossU$, thus

³⁸ $nmcpb$ stands for number-multiple-channel-partial-burn values of y .

³⁹ There are at least $2*nmctb$ such channels because there are $nmctb$ such target values y , $0 \leq y < P$, and for each such value of y there are at least 2 active channels with effective guess equal to y .

⁴⁰ There are at least $nmcpb$ such channels because there are $nmcpb$ such target values y , $0 \leq y < P$, and for each such value of y there is at least 1 active channel with effective guess equal to y such that a different channel will be put on-chain if the target value is y .

⁴¹ Note that $LossU$ and $LossT$ being analyzed here are for the depot protocol only, so the expected loss due to burn outputs in Lightning channels is not included. Lightning channel burn outputs are analyzed as part of the off-chain Lightning protocol in Appendix F.

providing sufficient penalization to prevent such an attack. Similarly, Appendix D proved that if the users perform a failure-to-drain attack and if all parties follow the protocol after the Targ or Targ' transaction is put on-chain⁴², $MinFracLossU \leq FracLossU$, once again providing sufficient penalization to prevent such an attack.

This appendix analyzes the effects of attacks by O or by users that combine a failure-to-drain attack with further depot protocol violations after the failure-to-drain.

For the remainder of this appendix, let:

- $AttackLossU$ = the expected loss to users from the attack (as opposed to the no-attack case in which all parties follow the depot protocol after the Targ or Targ' transaction is put on-chain), and
- $AttackLossO$ = the expected loss to O from the attack (as opposed to the no-attack case in which all parties follow the depot protocol after the Targ or Targ' transaction is put on-chain).

E.1 Security Against Other Attacks By The Operator

This subsection analyzes the security against O's attacks on users after O performs a failure-to-drain attack. The goal is to show that for all such attacks by O, $AttackLossO \geq 0$ (so O doesn't expect to gain from such an attack) and $AttackLossO \geq GrO * AttackLossU$ (so O pays a sufficient penalty to prevent O from performing the attack if the attack grieves honest users). The analysis is divided into cases based on the number of real hits, where a *real hit* is a hit by a user who didn't drain (as opposed to a *fake hit* that is a hit created by O). Those cases rely on the following technical results.

Technical Results

The $OperatorChannelMin$ parameter gives a lower bound on the operator's balance in any active channel while the $Capacity(i, MaxAu)$ function gives an upper bound on a user u 's balance in u 's i -th oldest channel. This subsection proves that these two bounds are consistent in the sense that u having a balance of $Capacity(i, MaxAu)$ in a channel never prevents the operator from having a balance of $OperatorChannelMin$ in the same channel.

Recall that $f(x) = \frac{e^x - 1 - x}{x(e^x - 1)}$ and $OperatorChannelMin = \frac{D * (1 - MaxFracLossU)}{2 * P}$.

Lemma 2: For any $x > 0$, $f'(x) < 0$.

Proof: Let $fnum(x) = e^x - 1 - x$ and $fden(x) = x * (e^x - 1)$

and note that $fnum'(x) = e^x - 1$ and $fden'(x) = e^x - 1 + x * e^x$.

Therefore

42 Or if (in some or all cases) the users also fail to burn the Hit transaction's output 1 despite having multiple hits.

$$f'(x) = \frac{(e^x - 1) * x * (e^x - 1) - (e^x - 1 - x) * (e^x - 1 + x * e^x)}{x^2 * (e^x - 1)^2} \text{ which equals}$$

$$\frac{x * e^{2x} - 2 * x * e^x + x - e^{2x} + e^x - x * e^{2x} + e^x - 1 + x * e^x + x * e^x - x + x^2 * e^x}{x^2 * (e^x - 1)^2} \text{ which equals}$$

$$\frac{2 * e^x + x^2 * e^x - e^{2x} - 1}{x^2 * (e^x - 1)^2}.$$

Note that if $x > 0$ then the denominator is greater than 0.

Using the Taylor series expansion $e^{kx} = \sum_{i=0}^{\infty} \left(\frac{(kx)^i}{i!} \right)$

the numerator equals

$$\sum_{i=0}^{\infty} \left(\frac{2 * x^i}{i!} \right) + \sum_{i=0}^{\infty} \left(\frac{x^{i+2}}{i!} \right) - \sum_{i=0}^{\infty} \left(\frac{(2 * x)^i}{i!} \right) - 1 \text{ which equals}$$

$$\sum_{i=2}^{\infty} \left(\frac{2 * x^i}{i!} \right) + \sum_{i=0}^{\infty} \left(\frac{x^{i+2}}{i!} \right) - \sum_{i=2}^{\infty} \left(\frac{(2 * x)^i}{i!} \right) \text{ which equals}$$

$$\sum_{i=2}^{\infty} \left(\frac{2 * x^i}{i!} \right) + \sum_{i=2}^{\infty} \left(\frac{i * (i-1) * x^i}{i!} \right) - \sum_{i=2}^{\infty} \left(\frac{(2 * x)^i}{i!} \right) \text{ which equals}$$

$$\sum_{i=2}^{\infty} \left(\frac{2 * x^i + i * (i-1) * x^i - 2^i * x^i}{i!} \right) \text{ which equals}$$

$$\sum_{i=2}^{\infty} \left(\frac{x^i * (2 + i * (i-1) - 2^i)}{i!} \right) \text{ which equals}$$

$$\sum_{i=4}^{\infty} \left(\frac{x^i * (2 + i * (i-1) - 2^i)}{i!} \right) \text{ which is less than 0.}$$

Therefore, for any $x > 0$, $f'(x) < 0$. $\square$

Lemma 3: For any $x > 0$, $f(x) < 1/2$.

Proof: Using the Taylor series expansion

$$e^x = 1 + x + x^2/2 + x^3/6 + x^4/24 + \dots$$

yields

$$f(x) = \frac{e^x - 1 - x}{x(e^x - 1)} = \frac{x^2/2 + x^3/6 + x^4/24 + \dots}{x^2 + x^3/2 + x^4/6 + x^5/24 + \dots} = \left(\frac{1}{2} \right) * \left(\frac{1 + x/3 + x^2/12 + x^3/60 + \dots}{1 + x/2 + x^2/6 + x^3/24 + \dots} \right)$$

so for any $x > 0$, $f(x) < 1/2$. $\square$

Lemma 4: $Capacity(1, MaxAu) < \frac{D * MaxFracLossU}{2 * P}$.

Proof: Note

$$Capacity(1, MaxAu) = MaxB(1, MaxAu) = \left(\frac{D * MaxFracLossU * MaxAu}{MaxA} \right) * \left(g\left(\frac{MaxA}{MaxAu * P}\right) \right) = \left(\frac{D * MaxFracLossU}{P} \right) * \left(f\left(\frac{MaxA}{MaxAu * P}\right) \right).$$

Note that $\frac{MaxA}{MaxAu * P} > 0$, so it follows from Lemma 3 that

$$Capacity(1, MaxAu) = \left(\frac{D * MaxFracLossU}{P} \right) * \left(f\left(\frac{MaxA}{MaxAu * P}\right) \right) < \frac{D * MaxFracLossU}{2 * P}. \quad \square$$

Lemma 5: For any i , $2 \leq i \leq MaxAu$, $Capacity(i, MaxAu) < \frac{D * MaxFracLossU}{2 * P}$.

Proof: Recall that for any i , $2 \leq i \leq MaxAu$,

$$Capacity(i, MaxAu) = \max(UserChannelMin, MaxB(i, MaxAu) - MaxB(i-1, MaxAu)).$$

There are two cases:

- **Case 1:** $Capacity(i, MaxAu) = UserChannelMin$. In this case, recall that $GrO * GrU < 1$ so

$$GrU + GrO * GrU < GrU + 1 \text{ and } \frac{GrU + GrO * GrU}{(1 + GrU) * (1 + GrO)} < \frac{GrU + 1}{(1 + GrU) * (1 + GrO)} \text{ and}$$

$$\frac{GrU}{1 + GrU} < \frac{1}{1 + GrO}, \text{ so } MinFracLossU < MaxFracLossU. \text{ Therefore,}$$

$$Capacity(i, MaxAu) = UserChannelMin = \frac{D * MinFracLossU}{2 * P} < \frac{D * MaxFracLossU}{2 * P}.$$

- **Case 2:** $Capacity(i, MaxAu) = MaxB(i, MaxAu) - MaxB(i-1, MaxAu)$. In this case, note that for any j , $1 \leq j \leq MaxAu$,

$$MaxB(j, MaxAu) = \left(\frac{D * MaxFracLossU * MaxAu}{MaxA} \right) * \left(g\left(\frac{j * MaxA}{MaxAu * P}\right) \right) =$$

$$\left(\frac{D * MaxFracLossU * j}{P} \right) * \left(f\left(\frac{j * MaxA}{MaxAu * P}\right) \right). \text{ Therefore, it follows from Lemma 2 that}$$

$$MaxB(i, MaxAu) = \left(\frac{D * MaxFracLossU * i}{P} \right) * \left(f\left(\frac{i * MaxA}{MaxAu * P}\right) \right) <$$

$$\left(\frac{D * MaxFracLossU * i}{P} \right) * \left(f\left(\frac{(i-1) * MaxA}{MaxAu * P}\right) \right). \text{ Therefore}$$

$$Capacity(i, MaxAu) = MaxB(i, MaxAu) - MaxB(i-1, MaxAu) <$$

$$\left(\frac{D * \text{MaxFracLossU} * (i - (i - 1))}{P}\right) * \left(f\left(\frac{(i - 1) * \text{MaxA}}{\text{MaxAu} * P}\right)\right) =$$

$$\left(\frac{D * \text{MaxFracLossU}}{P}\right) * \left(f\left(\frac{(i - 1) * \text{MaxA}}{\text{MaxAu} * P}\right)\right). \text{ Note that } \frac{(i - 1) * \text{MaxA}}{\text{MaxAu} * P} > 0, \text{ so it follows from}$$

Lemma 3 that $\text{Capacity}(i, \text{MaxAu}) < \frac{D * \text{MaxFracLossU}}{2 * P}.$

Thus, in either case, the lemma holds. $\square$

Combining Lemmas 4 and 5 yields Theorem 3.

Theorem 3: For any $i, 1 \leq i \leq \text{MaxAu}$, $\text{Capacity}(i, \text{MaxAu}) < \frac{D * \text{MaxFracLossU}}{2 * P}.$

Theorem 3 can be used to analyze other attacks by the operator based on the number of real hits.

Case 1: 0 Real Hits

In this case, if the protocol is followed O receives the entire value of the depot. Therefore, regardless of the nature of O's attack, $\text{AttackLossO} \geq 0$ and $\text{AttackLossU} \leq 0$, so $\text{AttackLossO} \geq \text{GrO} * \text{AttackLossU}.$

Case 2: Exactly 1 Real Hit

In this case, assume user u has the real hit where the channel with the hit has a payout to u of X so if O followed the protocol, u would receive X and O would receive $D - X$. O can violate the protocol in the following ways:

- **A) O puts a Hit transaction with a fake hit on-chain.** In this case u will use O's fake hit and u 's real hit to burn both of the Hit transaction's outputs. The result of this attack by O is thus $\text{AttackLossO} = D - X$ and $\text{AttackLossU} = X$. Note that $\text{AttackLossO} \geq 0$. It follows from Theorem 3 that $X < D * \text{MaxFracLossU} / 2$, so

$$\frac{\text{AttackLossO}}{\text{AttackLossU}} = \frac{D - X}{X} = \frac{D}{X} - 1 > \frac{D * 2}{D * \text{MaxFracLossU}} - 1 = 2 * (1 + \text{GrO}) - 1 = 2 * \text{GrO} + 1 > \text{GrO}$$

and $\text{AttackLossO} \geq \text{GrO} * \text{AttackLossU}.$

- **B) O puts an Arb_Oi transaction on-chain that spends the Hit transaction's output 1.** In this case, O reveals a guess signature $\text{ArbGuSig}_i \neq \text{GuSig}_i$, so u uses these two guess signatures to burn the Hit transaction's output 0. In this case, $\text{AttackLossO} = D/2 - X$ and $\text{AttackLossU} = X$. Note that $\text{AttackLossO} \geq 0$. It follows from Theorem 3 that $X < D * \text{MaxFracLossU} / 2$, so

$$\frac{\text{AttackLossO}}{\text{AttackLossU}} = \frac{D/2 - X}{X} = \frac{D}{2 * X} - 1 > \frac{D * 2}{2 * D * \text{MaxFracLossU}} - 1 = 1 + \text{GrO} - 1 = \text{GrO}$$

and $\text{AttackLossO} \geq \text{GrO} * \text{AttackLossU}.$

- **C) O puts a Burn transaction on-chain that burns the Clm_Ui transaction's output.** In this case, $AttackLossO = D - X - D/2 = D/2 - X$ and $AttackLossU = X$, so the analysis exactly matches the analysis of possibility B) above.
- **D) O puts a Clm_Vj transaction for a fake hit on-chain.** In this case, Clm_Vj reveals a guess signature $GuSig_j \neq GuSig_i$, so u uses these two guess signatures to burn the Clm_Vj output and the Hit transaction's output 0. The remaining analysis matches the analysis of possibility A) above.

Case 3: 2 Or More Real Hits

In this case, the only way O can violate the protocol after the Targ or Targ' transaction is put on-chain is if O puts a Hit transaction with a fake hit on-chain. However, in this case the users will burn both of the Hit transaction's outputs, which exactly matches the result when O follows the protocol. As a result, for this attack $AttackLossO = 0$ and $AttackLossU = 0$, so $AttackLossO \geq 0$ and $AttackLossO \geq GrO * AttackLossU$.

Conclusion

Appendix C proved that failure-to-drain attacks by O sufficiently penalize O, provided that O follows the protocol after the Targ or Targ' transaction is put on-chain. This subsection proved that any attack by O after the Targ or Targ' transaction is put on-chain is also sufficiently penalized. As a result, O is sufficiently penalized to prevent any attack by O on users.

Furthermore, even if some users fail to drain (either intentionally or due to temporary user failures), those users can expect O to follow the protocol after the Targ or Targ' transaction is put on-chain, as O has not motivation for violating the protocol.

E.2 Security Against Other Attacks By Users

This subsection analyzes the security against users' attacks on O after users perform a failure-to-drain attack. The goal is to show that for all such attacks by users, a failure-to-drain attack plus any later attack does not results in a gain for users and users are sufficiently penalized for any loss to O caused by such attacks.

Note that if $AttackLossU \geq 0$, users don't expect to gain from violating the protocol after the depot's expiry. Also, if $AttackLossU \geq GrU * AttackLossO$, users pay a sufficient penalty to prevent them from violating the protocol after the depot's expiry. Once again the analysis is divided into different cases based on the number of real hits.

Case 1: 0 Real Hits

In this case, O spends the output of the Targ or Targ' transaction, so the users cannot perform any attack on O.

Case 2: 1 Real Hit

In this case, let u be the user with the real hit. There are two possibilities:

- **A) User u fails to put a Hit transaction on-chain.** In this case O will spend the output of the Targ or Targ' transaction, so for this "attack" $AttackLossO < 0$ (as O gains the channel payment that should have gone to u) and $AttackLossU > 0$ (as u loses their channel payment), so $AttackLossU \geq 0$ and $AttackLossU \geq GrU * AttackLossO$.
- **B) User u fails to put a Clm_Ui transaction on-chain.** In this case, O waits $3 * tsd$ from when the Targ or Targ' transaction is put on-chain, then O spends the Hit transaction's output 0. After successfully spending the Hit transaction's output 0, O then puts an Arb_Oi transaction on-chain. This "attack" results in O obtaining all the funds in the depot, so the analysis matches possibility A) above.

Case 2: 2 Or More Real Hits

In this case, if the users followed the protocol all depot funds would be burned. There are several possibilities:

- **A) Users fail to put a Hit transaction on-chain.** In this case, O spends the output of the Targ or Targ' transaction and gains all of the depot's funds, so the "attack" results in $AttackLossO < 0$ and $AttackLossU = 0$, so $AttackLossU \geq 0$ and $AttackLossU \geq GrU * AttackLossO$.
- **B) Users put a Hit transaction on-chain and burn the Hit transaction's output 0.** In this case, either:
 - **B1): Users burn the Hit transaction's output 1.** In this case, all of the depot's funds are burned, so $AttackLossO = 0$ and $AttackLossU = 0$, which implies $AttackLossU \geq 0$ and $AttackLossU \geq GrU * AttackLossO$.
 - **B2): O spends the Hit transaction's output 1 with an Arb_Oi transaction.** In this case, O gains half of the depot's funds and users gain nothing, so $AttackLossO < 0$ and $AttackLossU \geq 0$, so $AttackLossU \geq GrU * AttackLossO$.
 - **B3): A user puts a Clm_Ui or Clm_Vj transaction on-chain and its output is burned.** In this case, all of the depot's funds are burned, so the analysis matches possibility B1) above.
 - **B4): A user puts a Clm_Ui or Clm_Vj transaction on-chain and its output is not burned.** If the operator followed the protocol and allowed users to drain, this case requires that users performed a failure-to-drain attack on the operator and then the users further violated the protocol by putting a channel on-chain despite having multiple hits. This case was analyzed in Appendix D where it was shown to result in an expected loss to the users that is at least GrU times the operator's expected loss. As a result, the users are sufficiently penalized to prevent such an attack on an operator who follows the protocol. (On the other

hand, if the operator didn't allow the users to drain, the operator failed to follow the protocol, so any loss to the operator is due to the operator's actions.)

- **C) Users put a Hit transaction on-chain but fail to burn the Hit transaction's output 0.**

This case matches possibility B) above, except the operator gains an additional $D/2$ relative to the corresponding case in possibility B). As a result, the users do not gain from such an attack and/or the users are sufficiently penalized if the attack grieves the operator.

Conclusion

Combined with the results of Appendix D, this subsection proves that failure-to-drain attacks by users sufficiently penalize the users, regardless of whether or not the users follow the protocol after the Targ or Targ' transaction is put on-chain.

Appendix F. Security Of The Off-Chain Lightning Protocol

This appendix gives formulas for determining the operator's and users' burn contributions for depot sends and receives and it proves that those burn contributions secure the off-chain Lightning protocol against griefing attacks.

F.1 Size Of Burn Outputs

At any given time assume the burn outputs for a user U's channels pay $BurnT$ sats and are funded by a reduction of O's outputs by $BurnO$ sats and U's outputs by $BurnU$ sats, where $BurnT = BurnO + BurnU$.

It follows from Theorem 1 that as long as $MinFracLossU \leq \frac{BurnU}{BurnT} \leq MaxFracLossU$ both the

operator and the user should be sufficiently deterred from griefing their partner. Therefore, the size of the burn outputs and the operator's and user's contribution to the burn outputs will be designed to

maintain the relationships $MinFracLossU \leq \frac{BurnU}{BurnT} \leq MaxFracLossU$ whenever a send or receive is being processed.

F.2 Lightning Payment From U

Consider a Lightning payment of X sats sent from U. If successful, this payment will decrease U's

outputs by $Y = X * P$ sats. In order to guarantee that $MinFracLossU \leq \frac{BurnU}{BurnT} \leq MaxFracLossU$

throughout the processing of the payment, $BurnT$ is set to $\frac{Y}{MaxFracLossU - MinFracLossU}$ which implies $Y = BurnT * (MaxFracLossU - MinFracLossU)$.

Before the payment, U contributes $BurnT * MaxFracLossU$ to the burn outputs and O contributes the remaining $BurnT * (1 - MaxFracLossU)$ to the burn outputs. At this point, the fraction of the expected loss to U from the burn outputs is:

$$\text{FracLossU} = \frac{\text{BurnU}}{\text{BurnT}} = \frac{\text{BurnT} * \text{MaxFracLossU}}{\text{BurnT}} = \text{MaxFracLossU}.$$

If the payment is successfully sent, U's contribution to the burn outputs will be reduced to $\text{BurnT} * \text{MaxFracLossU} - Y$, at which point the fraction of the expected loss to U from the burn outputs is:

$$\text{FracLossU} = \frac{\text{BurnT} * \text{MaxFracLossU} - \text{BurnT} * (\text{MaxFracLossU} - \text{MinFracLossU})}{\text{BurnT}} = \text{MinFracLossU}.$$

On the other hand, if the payment fails, O's and U's contributions to the burn outputs are refunded to their outputs.

F.3 Lightning Payment To U

The case where U receives a Lightning payment of X sats is the same process in reverse. If successful, this payment will increase U's outputs by $Y = X * P$ sats. In order to guarantee that

$$\text{MinFracLossU} \leq \frac{\text{BurnU}}{\text{BurnT}} \leq \text{MaxFracLossU} \quad \text{throughout the payment, } \text{BurnT} \text{ is again set to}$$

$$\frac{Y}{\text{MaxFracLossU} - \text{MinFracLossU}} \quad \text{which implies}$$

$$Y = \text{BurnT} * (\text{MaxFracLossU} - \text{MinFracLossU}).$$

Before the payment, U contributes $\text{BurnT} * \text{MinFracLossU}$ to the burn outputs and O contributes the remaining $\text{BurnT} * (1 - \text{MinFracLossU})$ to the burn outputs. At this point, the fraction of the expected loss to U from the burn outputs is:

$$\text{FracLossU} = \frac{\text{BurnU}}{\text{BurnT}} = \frac{\text{BurnT} * \text{MinFracLossU}}{\text{BurnT}} = \text{MinFracLossU}.$$

If the payment is successfully received, U's contribution to the burn outputs will be increased to $\text{BurnT} * \text{MinFracLossU} + Y$, so the fraction of the expected loss to U from the burn outputs is:

$$\text{FracLossU} = \frac{\text{BurnT} * \text{MinFracLossU} + \text{BurnT} * (\text{MaxFracLossU} - \text{MinFracLossU})}{\text{BurnT}} = \text{MaxFracLossU}.$$

On the other hand, if the payment fails, O's and U's contributions to the burn outputs are refunded to their outputs.

Appendix G. Quantification Of Burn Depots

This appendix provides a quantitative analysis⁴³ of burn depots using the notation introduced in Appendix B.

43 The program `burn_depot.py` [Law26] can be used to calculate the expected amount burned for an arbitrary burn depot, as well as the uncertainty in the expected amount burned and how that uncertainty can be tolerated with matching contributions.

G.1 Expected Amount Burned Per Active Channel

A burn depot with A active channels, each of which has a probability of $1/P$ of being a hit, has $H = A/P$

expected hits. The Poisson approximation $p(k; H) = \frac{H^k}{k! * e^H}$ can be used to calculate the probability of exactly k hits, yielding a probability of:

- $\frac{1}{e^H}$ of 0 hits, in which case nothing is burned,
- $\frac{H}{e^H}$ of exactly 1 hit, in which case $D/2$ is burned, and
- $1 - \frac{1+H}{e^H} = \frac{e^H - 1 - H}{e^H}$ of 2 or more hits, in which case D is burned.

As a result, the expected amount burned is $\frac{(H/2) + e^H - 1 - H}{e^H} * D = \frac{e^H - 1 - (H/2)}{e^H} * D$ and the

expected amount burned per active channel is $\frac{e^H - 1 - (H/2)}{A * e^H} * D = \frac{e^H - 1 - (H/2)}{H * e^H} * \frac{D}{P} = b(H) * \frac{D}{P}$

where $b(x) = \frac{e^x - 1 - (x/2)}{x * e^x}$.

G.2 Burn Depots With Uncertain Numbers Of Active Channels

If a user u knew the number of active channels in a burn depot, they could use the result from Appendix G.1 to determine the expected amount burned by their A_u active channels. However, user u only has an upper bound, $MaxA$, on the possible number of channels that are active in a burn depot. The following Lemma shows how the upper bound on active channels can be used to obtain a lower bound on the expected amount burned per channel.

Lemma 6: For any $x > 0$, $b'(x) < 0$.

Proof: Let $bnum(x) = e^x - 1 - x/2$ and $bden(x) = x * e^x$

and note that $bnum'(x) = e^x - 1/2$ and $bden'(x) = e^x + x * e^x$.

Therefore

$$b'(x) = \frac{(e^x - 1/2) * x * e^x - (e^x - 1 - x/2) * (e^x + x * e^x)}{x^2 * e^{2x}} \text{ which equals}$$

$$\frac{x * e^{2x} - (x/2) * e^x - e^{2x} - x * e^{2x} + e^x + x * e^x + (x/2) * e^x + (x^2/2) * e^x}{x^2 * e^{2x}} \text{ which equals}$$

$$\frac{-e^{2x} + e^x + x * e^x + (x^2/2) * e^x}{x^2 * e^{2x}} \text{ which equals}$$

$$\frac{-e^x + 1 + x + x^2/2}{x^2 * e^x}.$$

Note that if $x > 0$ then the denominator is greater than 0.

Using the Taylor series expansion $e^{kx} = \sum_{i=0}^{\infty} \frac{(kx)^i}{i!}$

the numerator equals $1 + x + x^2/2 - \sum_{i=0}^{\infty} \frac{x^i}{i!}$ which equals $-\sum_{i=3}^{\infty} \frac{x^i}{i!}$ which is less than 0.

Therefore, for any $x > 0$, $b'(x) < 0$. $\square$

As a result of Lemma 6, if user u wants to obtain active channels in a burn depot that are expected to

burn at least X , u requires $Au \geq \frac{X * P}{b(MaxH) * D} = \frac{X * P}{b(MaxA/P) * D}$ active channels.

G.3 Charging For Uncertainty

While providing user u with $Au \geq \frac{X * P}{b(MaxA/P) * D}$ active channels guarantees that u 's channels are

expected to burn at least X in the burn depot, there's a risk to O that O won't be able to sell all $MaxA$ channels in the burn depot, thus causing O to burn an excessive amount. This is problematic because it increases O's expected loss without affecting user u 's expected loss, potentially violating the security

guarantee that $\frac{LossU}{LossO} \geq GrU$.

To prevent this possibility, O determines a lower bound, $MinSaleFrac$, on the fraction of the $MaxA$ allocated channels that O can sell in the burn depot. Given this lower bound, the additional loss to U per active channel is at most

$$(b(MinSaleFrac * MaxH) - b(MaxH)) * \frac{D}{P} = PerChannelUncertaintyBound. \text{ Therefore, user } u \text{ and}$$

O are required to burn enough matching funds in the burn depot to guarantee that any additional loss to U cannot exceed the required security guarantees. In particular, if user u purchases Au active channels in the burn depot, those Au channels could burn an excess of up to $Y =$

$Au * PerChannelUncertaintyBound$ sats. Therefore, u and O burn matching funds, $Mu(Y)$ and $Mo(Y)$ respectively, where:

- $Mu(Y) = Y * \frac{MaxFracLossU * MinFracLossU}{MaxFracLossU - MinFracLossU}$ and

- $Mo(Y) = Y * \frac{(1 - MaxFracLossU) * MinFracLossU}{MaxFracLossU - MinFracLossU}$.

To see that these matching funds are sufficient, let X denote the actual excess expected burn value of the Au channels purchased by u , and note that $0 \leq X \leq Y$. Let $LossU$ denote user u 's expected additional loss due to the use of a burn depot, let $LossT$ denote the total expected additional loss to u and O due to the use of a burn depot to burn u 's active channels, and let $FracLossU = LossU/LossT$. Note that:

- $LossU = Mu(Y) = Y * \frac{MaxFracLossU * MinFracLossU}{MaxFracLossU - MinFracLossU}$ and
- $LossT = X + Mu(Y) + Mo(Y) = X + Y * \frac{MinFracLossU}{MaxFracLossU - MinFracLossU}$.

Because $0 \leq X \leq Y$, it follows that $Y * \frac{MinFracLossU}{MaxFracLossU - MinFracLossU} \leq LossT$ and

$$LossT \leq Y + Y * \frac{MinFracLossU}{MaxFracLossU - MinFracLossU} = Y * \frac{MaxFracLossU}{MaxFracLossU - MinFracLossU}.$$

Therefore, $FracLossU \geq \frac{MaxFracLossU * MinFracLossU}{MaxFracLossU} = MinFracLossU$ and

$$FracLossU \leq \frac{MaxFracLossU * MinFracLossU}{MinFracLossU} = MaxFracLossU.$$

For example⁴⁴, assume:

- $GrO = 0.05$,
- $GrU = 0.30$,
- $D = 1,864,361$ sats,
- $P = 1,864,361$,
- $MaxH = 1.00$, and
- $MinSaleFrac = 0.90$.

In this case, each Lightning channel is expected to burn at least

$$b(MaxH) * \frac{D}{P} = b(1.00) * \frac{1864361}{1864361} \approx 0.448181 \text{ sats and at most}$$

$$b(MinSaleFrac * MaxH) * \frac{D}{P} = b(0.90) * \frac{1864361}{1864361} \approx 0.456082 \text{ sats, yielding an uncertainty of}$$

0.007901 sats per channel (which is about 1.763% of the required burn value per channel). Thus, if user

44 The program burn_depot.py [Law26] was used to make all of the calculations for this example.

u gets $Au = 10,000$ channels with an expected burn amount of at least 4,481.81 sats, there is an uncertainty of 79.01 sats in the expected amount burned by these 10,000 channels.

Because $MinFracLossU \approx 0.230769$ and $MaxFracLossU \approx 0.952381$, for each 1 sat of uncertainty in the expected value of the amount burned user u burns an additional matching

$$Mu(1) = 1 * \frac{MaxFracLossU * MinFracLossU}{MaxFracLossU - MinFracLossU} \approx \frac{0.219780}{0.721612} \approx 0.304569 \text{ sats}$$

and O burns an additional matching

$$Mo(1) = 1 * \frac{(1.0 - MaxFracLossU) * MinFracLossU}{MaxFracLossU - MinFracLossU} \approx \frac{0.010989}{0.721612} \approx 0.015228 \text{ sats, for a total of about}$$

0.319797 in matching sats, which in turn are implemented with channels that have an uncertainty of about $0.319797 * 0.0177 \approx 0.005660$ sats, which in turn requires additional matching contributions, etc. Thus each sat of uncertainty in the amount burned for user u 's Au channels thus yields a total uncertainty of about $1/(1-0.005660) \approx 1.005693$ sats burned. Therefore, the total uncertainty in the amount burned for user u 's Au channels plus matching burn channels is about $1.005693 * 79.01 \approx 79.46$ sats. To accommodate this uncertainty, user u burns an additional matching

$$Mu(Y) = Y * \frac{MaxFracLossU * MinFracLossU}{MaxFracLossU - MinFracLossU} \approx 79.46 * \frac{0.219780}{0.721612} \approx 79.46 * 0.304569 \approx 24.20$$

sats and O burns an additional matching

$$Mo(Y) = Y * \frac{(1.0 - MaxFracLossU) * MinFracLossU}{MaxFracLossU - MinFracLossU} \approx 79.46 * \frac{0.010989}{0.721612} \approx 79.46 * 0.015228 \approx 1.21$$

sats. These matching burn contributions are implemented using $\text{ceil}((24.20 + 1.21)/0.448181) = 57$ channels in the burn depot, for a total of 10,057 channels in the burn depot. User u 's matching contribution is made by transferring 24.20 sats to O in the original depot with the burn outputs that are being removed, while O's matching contribution comes from O granting an additional 57 channels to user u in the burn depot.

If O is able to sell the full $MaxA$ active channels in the burn depot, each channel burns an expected

$$0.448181 \text{ sats, so } FracLossU = \frac{LossU}{LossT} \approx \frac{24.20}{57 * 0.448181} \approx 0.9473 < MaxFracLossU. \text{ At the other}$$

extreme, if O is only able to sell $MinSaleFrac * MaxA$ active channels in the burn depot, each channel burns an expected 0.456082 sats, so

$$FracLossU = \frac{LossU}{LossT} \approx \frac{24.20}{79.01 + 57 * 0.456082} \approx 0.2305 \approx MinFracLossU.$$

Appendix H. Script Subroutines

The body of this paper uses the multiplication (OP_MUL), mod (OP_MOD) and PC (PairCommit) operators. This appendix shows how the required functionality can be achieved without those operators.

H.1 Effective Guess Calculation

In order to determine if channel i is a hit, one must calculate its effective guess $(\bar{R} * \bar{F}_i + G_i) \bmod P$.

Each multiplication $r_x * f_{ix}$ within the inner product $\bar{R} * \bar{F}_i$ can be implemented with repeated doublings and additions and the mod P operation can be implemented by subtracting the value P from the current subtotal whenever it exceeds P :

Let $C = 2^{\text{floor}(\log_2 P)}$ be the largest power of 2 that is less than or equal to P .

The following Script program shows how $(\bar{R} * \bar{F}_i + G_i) \bmod P$ can be evaluated using only the currently-supported Script operands.

```
// Unoptimized Script program to calculate  $(\bar{R} * \bar{F}_i + G_i) \bmod P$ 
// stack: <r_g> <f_ig> ... <r_2> <f_i2> <r_1> <f_i1> <G_i> <P>
  2SWAP
// stack: <r_g> <f_ig> ... <r_2> <f_i2> <G_i> <P> <r_1> <f_i1>
// multiply f_i1 by most significant bit of r_1:
  SWAP <C> 2DUP GREATERTHANOREQUAL
  IF
    SUB SWAP DUP ADD <2> PICK 2DUP GREATERTHANOREQUAL
    IF
      SUB
    ELSE
      DROP
    ENDIF
// stack: <f_ig> <r_g> ... <f_i2> <r_2> <G_i> <P> <r_1-C> <2*f_i1 mod P>
  ELSE
    DROP SWAP
// stack: <f_ig> <r_g> ... <f_i2> <r_2> <G_i> <P> <r_1> <f_i1>
  ENDIF
// multiply f_i1 by second most significant bit of r_1:
  SWAP <C/2> 2DUP GREATERTHANOREQUAL
  IF
    SUB SWAP DUP ADD <2> PICK 2DUP GREATERTHANOREQUAL
```

```

    IF
        SUB
    ELSE
        DROP
    ENDIF
ELSE
    DROP SWAP
ENDIF
// repeat through least significant bit of r_1:
...
// multiply f_i1 by least significant bit of f_1:
SWAP <1> 2DUP GREATERTHANOREQUAL
IF
    SUB SWAP DUP ADD <2> PICK 2DUP GREATERTHANOREQUAL
    IF
        SUB
    ELSE
        DROP
    ENDIF
ELSE
    DROP SWAP
ENDIF
// stack: <f_ig> <r_g> ... <f_i2> <r_2> <G_i> <P> <0> <r_1*f_i1 mod P>
NIP <2> ROLL
// stack: <f_ig> <r_g> ... <f_i2> <r_2> <P> <r_1*f_i1 mod P> <G_i>
ADD OVER 2DUP GREATERTHANOREQUAL
IF
    SUB
ELSE
    DROP
ENDIF
// stack: <f_ig> <r_g> ... <f_i2> <r_2> <P> <(r_1*f_i1 + G_i) mod P>
// repeat for r_2*f_i2 through r_g*f_ig
...
// stack: <P> < $\bar{R} \cdot \bar{F}_i + G_i \text{ mod } P$ >
NIP
// stack: < $\bar{R} \cdot \bar{F}_i + G_i \text{ mod } P$ >

```

Note that $0 \leq r_x < P$ for $1 \leq x \leq g$ and g is the smallest integer such that $P^g \geq 2^{512}$. Therefore, the total length of the above script is constant regardless of the value of P .

H.2 CSFS Signatures Without The PC Operator

The CSFS operator is used to sign targets, channel indices and guesses. CSFS signatures for targets and guesses sign a list of values, despite the fact that the CSFS operator only signs a single stack element. If the PC operator is available, it's used repeatedly to obtain a single commitment for the list of values.

If the PC operator is unavailable, Black and Rubin's beautiful *key laddering* technique [BR24] is used to create target signatures.

Signing guesses without the PC operator relies on a modified version of key laddering. Black and Rubin's technique assumes the parties creating the CSFS signature know all the values being signed. Unfortunately, the operator cannot know all the values signed by a guess signature. In particular, the protocol presented in the body of this paper requires that O signs for the vector $\langle Clm_Ui, Clm'_Ui, s_i, \bar{F}_i, UO, i, InSig_i, G_i \rangle$, but O cannot know G_i . Furthermore, O cannot just sign for $\langle Clm_Ui, Clm'_Ui, s_i, \bar{F}_i, UO, i, InSig_i \rangle$, as there is no way to make this signature connect with G_i without using the PC operator.

Instead, when the PC operator is unavailable, a different guess signature is used. Rather than signing for $\langle Clm_Ui, Clm'_Ui, s_i, \bar{F}_i, UO, i, InSig_i, G_i \rangle$, O signs for $\langle Clm_Ui, Clm'_Ui, s_i, \bar{F}_i, U, HSSU_i \rangle$, where $HSSU_i = \text{SHA256}(SSU_i)$ and SSU_i satisfies $\text{CSFS}(U, G_i, SSU_i)$.

The complete protocol for creating the guess signature is as follows. First, U selects a random guess G_i and creates the signature SSU_i that satisfies $\text{CSFS}(U, G_i, SSU_i)$. Then U gives O the value $HSSU_i = \text{SHA256}(SSU_i)$ and O uses key laddering to create the guess signature $GuSig_i$ that satisfies $\text{CSFS}(O, \langle Clm_Ui, Clm'_Ui, s_i, \bar{F}_i, U, HSSU_i \rangle, GuSig_i)$. The Script for claiming that channel i is a hit checks $\text{CSFS}(O, \langle Clm_Ui, Clm'_Ui, s_i, \bar{F}_i, U, HSSU_i \rangle, GuSig_i)$ and $HSSU_i = \text{SHA256}(SSU_i)$ and $\text{CSFS}(U, G_i, SSU_i)$. Note that U 's selection of a nonce when calculating SSU_i and the use of the hash function in calculating $HSSU_i$ prevent O from determining the value of G_i by grinding through all possible value for the guess. Also, note that U must remember both the guess G_i and the signature SSU_i unless and until U revokes the channel by revealing the guess and signature to O .

Finally, note that the secret that O needs to be able to grind through all possibilities to calculate the guess G_i is the hash preimage of $HSSU_i$, namely SSU_i . As a result, in order to make the revocation of channel i atomic with a Lightning payment from O to U (as described in Section 3.5), the Lightning payment must use HTLCs with the secret SSU_i . This is awkward, as the granting of channels requires a Lightning payment that uses PTLCs (rather than HTLCs). Avoiding these complications, plus the larger script that results from key laddering, provides a strong case for supporting the PC operator when implementing depots.

Appendix I. Determining The Number Of Targets

Targets are expensive, as O has to put a target value t_y on-chain for each one. However, if O sells too many channels for a given target, O runs the risk of being unable to select a target value that misses all of the revoked channels. This appendix analyzes how many channels can be sold for a given target without creating any realistic risk for O.

Quantifying this risk requires the use of special numbers known as *Stirling numbers of the second kind* (see Section 6.1, pp. 257-267 of [GKP94]).

Notation:

For any $n \geq 0$ and $m \geq 0$, let $\begin{Bmatrix} n \\ m \end{Bmatrix}$ denote the *Stirling number of the second kind* with parameters n and m , which is defined as the number of ways to partition n identifiable items into m nonempty subsets.

For example, there are 7 ways to partition 4 items into 2 nonempty subsets:

$$\{1,2,3\} \cup \{4\}, \quad \{1,2,4\} \cup \{3\}, \quad \{1,3,4\} \cup \{2\}, \quad \{2,3,4\} \cup \{1\}, \quad \{1,2\} \cup \{3,4\}, \quad \{1,3\} \cup \{2,4\}, \\ \{1,4\} \cup \{2,3\}.$$

Stirling numbers of the second kind can be calculated using the recurrence:

$$\begin{Bmatrix} n \\ m \end{Bmatrix} = m * \begin{Bmatrix} n-1 \\ m \end{Bmatrix} + \begin{Bmatrix} n-1 \\ m-1 \end{Bmatrix},$$

where the first term on the right-hand side captures the number of ways in which one can add the n -th item to a nonempty subset of the first $n-1$ items and the second term on the right-hand side captures the number of ways in which one can put the n -th item into its own subset.

The base cases are:

$$\begin{Bmatrix} 0 \\ 0 \end{Bmatrix} = 1 \text{ (by convention), } \begin{Bmatrix} n \\ 0 \end{Bmatrix} = 0 \text{ for } n \geq 1, \quad \begin{Bmatrix} 0 \\ 1 \end{Bmatrix} = 0 \text{ and } \begin{Bmatrix} n \\ 1 \end{Bmatrix} = 1 \text{ for } n \geq 1.$$

Let $Burn(r, P)$ denote the probability that r revoked channels hit all P possible values for a target and let $FirstBurn(q, P)$ denote the probability that it requires exactly q revoked channels to hit all P possible values for the first time.

Note that it takes at least P revoked channels to hit all P values, so $Burn(r, P) = \sum_{q=P}^r FirstBurn(q, P)$.

For any $q \geq P$, $FirstBurn(q, P) = P^{-q} * P! * \begin{Bmatrix} q-1 \\ P-1 \end{Bmatrix}$ (see Answer to Exercise 8.38, p.583 of [GKP94], where this is called the *coupon collecting* problem)⁴⁵.

⁴⁵ To see why this is true, assume that it requires exactly q revoked channels to hit all P possible values for the first time. In this case, the first $q-1$ revoked channels must have hit exactly $P-1$ of the

Theorem 4: If users revoke r channels for a single target with P possible values, the probability that these revoked channels have effective guesses that match all P possible target values equals

$\sum_{q=P}^r P^{-q} * P! * \left\{ \begin{matrix} q-1 \\ P-1 \end{matrix} \right\}$ where $\left\{ \begin{matrix} q-1 \\ P-1 \end{matrix} \right\}$ is the Stirling number of the second kind with parameters $q-1$ and $P-1$.

Proof: Follows directly from the fact that the effective values of the channels are selected uniformly and independently, the fact that users have no knowledge of a channel's effective value when they choose to revoke the channel, and the Answer to Exercise 8.38, p.583 of [GKP94]. $\square$

The following table uses Theorem 4 to calculate the probability that r revoked channels will have effective guesses that match all $P = 500$ possible target values for different values of r^{46} .

r	r/P	Probability r channels have effective guesses that match all $P = 500$ values
500	1	$3.9939844265475195 * 10^{-216}$
1,000	2	$1.5554198972414202 * 10^{-39}$
1,500	3	$8.303616420703816 * 10^{-13}$
2,000	4	$6.953884482684047 * 10^{-5}$
2,500	5	0.03264909638260816
3,000	6	0.28860145881685134
3,500	7	0.6348287824114646
4,000	8	0.8465029634744793
4,500	9	0.9406458960772185
5,000	10	0.9777714628337447
5,500	11	0.9917738364709568
6,000	12	0.9969690713095982
6,500	13	0.9988850564829265
7,000	14	0.9995901049872037
7,500	15	0.9998493399952596
8,000	16	0.9999446281946502
8,500	17	0.9999796498934612

Table 8. Results from Theorem 4 for $P = 500$.

possible values, and the q -th channel must have hit the remaining value. There are $\left\{ \begin{matrix} q-1 \\ P-1 \end{matrix} \right\}$ ways

in which to partition the first $q-1$ revoked channels into $P-1$ nonempty subsets that hit the same value (ignoring what those values are) and there are $P!$ ways of assigning values to those subsets.

Finally, the probability of having any exact sequence of q values is P^{-q} .

⁴⁶ The program coupon.py [Law26] was used to generated the values in this table.

In principle, Theorem 4 could be used to determine the exact risk to the operator of selling r channels for a single target with P possible values for arbitrarily large values of P . However, in practice performing that calculation for values of P much greater than 500 involves extremely large and small values that can't be accurately represented with 64-bit floating point numbers. As a result, it's helpful to have a way to bound the probability that r channels will have effective guesses that match all P possible target values, where r and P are large. The following theorem provides such a bound.

Theorem 5: If r revoked channels for a single target with P possible values have probability x of having effective guesses matching all P possible target values, then for any integer $k \geq 1$, the probability that $k*r$ revoked channels for a single target with $k*P$ possible values have at most x^k probability of having effective guesses matching all $k*P$ possible target values.

Proof: Logically partition the $k*P$ possible target values into k sets of P values each and consider all possible assignments of the $k*r$ revoked channels to these k sets (without specifying the assignments of the channels to the specific target values within each set). It's clear that the probability that the $k*r$ channels match all $k*P$ possible target values is maximized when exactly r channels have a value in each of the k sets⁴⁷. Furthermore, if exactly r channels have a value in each of the k sets, the probability that any given set has channels that match all P values in the set is x and the probability that all k of the sets have channels that match all P values in the set is x^k . $\square$

The following table uses Theorem 5 and the results from Table 8 to provide values of $P = k*500$ that have at most a 10^{-30} probability that $k*r$ channels for a single target with P possible values will have effective guesses that match all P possible target values.

⁴⁷ To see this, assume for the sake of contradiction that there is a different mapping of channels to sets that has a greater probability of matching all $k*P$ possible target values. This different mapping must map fewer than r channels to some set i and more than r channels to some set j . However, if one of the channels mapped to set j is remapped to have a random value in set i , the resulting mapping will have a greater probability of matching all $k*P$ possible target values, which is a contradiction.

$r/500$	$x = \text{prob } r \text{ revoked channels hit all } 500 \text{ values}$	Smallest integer k s.t. $x^k \leq 10^{-30}$	$P = k*500$
1	$3.9939844265475195 * 10^{-216}$	1	500
2	$1.5554198972414202 * 10^{-39}$	1	500
3	$8.303616420703816 * 10^{-13}$	3	1,500
4	$6.953884482684047 * 10^{-5}$	8	4,000
5	0.03264909638260816	21	10,500
6	0.28860145881685134	56	28,000
7	0.6348287824114646	153	76,500
8	0.8465029634744793	415	207,500
9	0.9406458960772185	1,129	564,500
10	0.9777714628337447	3,073	1,536,500
11	0.9917738364709568	8,363	4,181,500
12	0.9969690713095982	22,757	11,378,500
13	0.9988850564829265	61,922	30,961,000
14	0.9995901049872037	168,491	84,245,500
15	0.9998493399952596	458,466	229,233,000
16	0.9999446281946502	1,247,506	623,753,000
17	0.9999796498934612	3,394,441	1,697,220,500

Table 9. Results from applying Theorem 5 to the results from Table 8.

For example, $r = 5,000$ channels have a probability of $x = 0.9777714628337447$ of hitting all $P = 500$ values (as was shown in Table 8). Therefore, $r * 3,073 = 15,365,000$ channels have a probability of at most $x^{3,073} = 9.986 \times 10^{-31} < 10^{-30}$ of hitting all $P = 3,073 * 500 = 1,536,500$ target values (from Theorem 5).

The results in Table 9 can be used to determine the number h of targets that are required for any value of $MaxC$ cumulative number of channels and P possible values per target (assuming the operator is willing to tolerate a 10^{-30} chance per target of having all of that target's values being hit by a revoked channel). For example, if $MaxC = 1,000,000,000$ and $P = 1,864,361$, then $P > 1,536,500$ so $r = 10 * P = 18,643,610$ channels can be sold per target, thus requiring a total of $h = \text{ceiling}(1,000,000,000/18,643,610) = 54$ targets.